

Exploring Multi-Modal Deep Learning: Enhancing Vision-based DML and OOD Detection through Language Integration

Submitted in partial fulfillment of the requirements
for the degree of

Bachelor of Technology in Electrical Engineering and
Master of Technology in Communication & Signal Processing

By

Vaibhav Nawal
(Roll No. 18D070033)

Under the guidance of
Prof. Biplab Banerjee



Department of Electrical Engineering
Indian Institute of Technology Bombay
2022-2023

Dissertation Approval

The dissertation entitled

**Exploring Multi-Modal Deep Learning: Enhancing Vision-based
DML and OOD Detection through Language Integration**

by

Vaibhav Nawal

is approved for the degree of

**Bachelor of Technology in Electrical Engineering and
Master of Technology in Communication & Signal Processing**

Digital Signature
Biplab Banerjee (10001772)
03-Aug-23 11:11:31 AM

Prof. Biplab Banerjee

Centre of Studies In Resources Engineering
(Supervisor)

Digital Signature
avik bhattacharya (i11038)
01-Aug-23 09:41:32 AM

Prof. Avik Bhattacharya

Centre of Studies In Resources Engineering
(Examiner)

Digital Signature
Karthikeyan Lanka (10001841)
01-Aug-23 02:21:49 AM

Prof. Karthikeyan Lanka

Centre of Studies in Resources Engineering
(Examiner)

Declaration

I declare that this written submission represents my ideas in my own words. I have cited and referenced the original sources where others' ideas or words have been included. I also declare that I have adhered to all academic honesty and integrity principles and have not misrepresented, fabricated, or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will result in disciplinary action by the Institute. In addition, it can also evoke penal action from sources that are not correctly cited or from whom proper permission has not been taken when needed.

Date: 28 June 2023

A handwritten signature in black ink, appearing to read 'Vaibhav Nawal', with a horizontal line extending to the right from the end of the signature.

Vaibhav Nawal

18D070033

Acknowledgement

I would like to extend my heartfelt gratitude to Prof. Biplab Banerjee for granting me the opportunity to undertake this project under his esteemed guidance. Throughout the duration of my work, his invaluable support and insightful suggestions have played a pivotal role in the project's success. I would also like to express my sincere appreciation to the department for generously providing me with all the necessary resources and facilities essential for the project. Finally thanking my friends and all my peers on the project for their help.

I would also like to acknowledge that this project has been a collaborative effort with Siddhesh Pradeep Sali (18D070030), and I would like to extend my gratitude for his significant contributions to this project.

Vaibhav Nawal
Electrical Engineering
IIT Bombay

Abstract

Deep learning models have predominantly relied on a single modality, either vision or language, for various tasks. This project aims to leverage the rich information present in other modalities, particularly language, to improve the performance of vision models in specific tasks. The research focuses on integrating language into computer vision tasks such as vision-based deep metric learning (DML) and out-of-distribution (OOD) detection, and builds upon recent advancements in these fields to further enhance the accuracy of the results.

In the first stage of the project, vision-based Deep Metric Learning is explored, where the incorporation of language guidance demonstrates significant improvements. Building upon this, a method called Hint Learning is proposed to enhance the language guidance technique and further improve the performance of DML model.

Moving to the second stage, Out-Of-Distribution Detection is investigated, and a technique called Maximum Concept Matching (MCM) is identified as a promising approach that leverages aligned visual features and textual concepts for accurate identification of OOD samples. This multi-modal perspective proves to be more effective than existing single-modal approaches. To enhance the results, Prompt Tuning or Prompt Learning Method is proposed in conjunction with the MCM method, aiming to refine the prompts used in the pre-trained models and potentially improve the model's accuracy.

The research demonstrates that language integration in vision-based deep metric learning and out-of-distribution detection can significantly enhance the performance of these models. By leveraging the complementary information provided by language, the learned embedding spaces in DML become more semantically meaningful and transferable to other classes. The multi-modal perspective in OOD detection improves the accuracy in identifying OOD samples, especially in challenging scenarios involving semantically similar classes.

Overall, this thesis contributes to the exploration and advancement of multi-modal deep learning, showcasing the benefits of integrating language into vision-based tasks and providing novel methods to improve the performance of deep metric learning and out-of-distribution detection models.

Code available at <https://github.com/sid4sal/LanguageGuidedVision> [1]

KEYWORDS: Out-of-distribution Detection (OOD Detection), Maximum Concept Matching (MCM), Multi-modal representations, Zero-shot, Few shot, Vision-language features, Prompt Learning, Prompt Tuning, Fine Tuning, Context Optimization (CoOp), CLIP model, Hard and Soft Prompts, Deep Metric Learning (DML), Knowledge Distillation, Hint Learning, Similarity Learning, Representation Learning, Computer Vision, Deep Learning, Machine Learning.

Contents

Abstract	v
Contents & List of Figures	vi
I Deep Metric Learning (DML)	1
1 Introduction: Multi-Modal DML	2
1.1 Motivation	2
1.2 Background	2
1.3 Contribution	3
1.4 Organization of the Report	3
2 Deep Metric Learning	4
2.1 Description of Deep Metric Learning	4
2.2 Applications of Deep Metric Learning	4
2.3 Methods used in Deep Metric Learning	5
2.3.1 Contrastive Loss (Siamese Networks)	5
2.3.2 Triplet Loss	7
2.3.3 Multi Similarity Loss	7
3 Language Guidance	10
3.1 Introduction to Language Guidance	10
3.2 Language Guidance Model Architecture	11
3.2.1 Expert Language Guidance (ELG)	12
3.2.2 Pseudolabel Language Guidance (PLG)	13
3.3 Results of Language Guidance	14
3.3.1 Comparison	14
3.3.2 Our Results and Conclusion	15
4 Hint Learning	17
4.1 Introduction to Hint Learning	17
4.2 Method	18

5	Results: DML	20
5.1	Results	20
5.2	Conclusion	21
5.3	Future Scope	22
II	Out-Of-Distribution (OOD) Detection	23
6	Introduction: Multi-Modal OOD Detection	24
6.1	Motivation	24
6.2	Background	25
6.3	Contribution	25
6.4	Organization of the Report	26
7	Out-Of-Distribution Detection	27
7.1	Fundamentals of Out-Of-Distribution Detection	27
7.2	Applications of Out-Of-Distribution Detection	27
7.3	Approaches for Out-Of-Distribution Detection	28
7.3.1	Maximum Softmax Probability (MaxProb or MSP)	28
7.3.2	Ensemble Learning	29
7.3.3	Temperature Scaling	29
7.3.4	ODIN (Out-of-Distribution Detector for Neural Networks)	30
7.3.5	SelectiveNet	30
7.4	Evaluation Metrics for Out-Of-Distribution Detection Methods	31
7.4.1	FAR95	31
7.4.2	AUROC	31
7.4.3	AUPR	32
7.4.4	FPR95	33
8	Maximum Concept Matching (MCM)	34
8.1	Description of MCM	34
8.2	Advantages of MCM	34
8.3	MCM Approach	35
8.4	Results: MCM	37
8.4.1	Our results	37
8.4.2	Comparison	40
8.4.3	Conclusion	41
9	Prompt Learning/Tuning	42
9.1	Introduction to Prompt Learning	42
9.2	Methods for Prompt Learning	43
9.2.1	Context Optimization (CoOp)	43

9.2.2 Conditional Context Optimization (CoCoOp)	44
10 Results: OOD Detection	46
10.1 Hyperparameter Selection	46
10.2 Training	47
10.3 Results	48
10.4 Conclusion	50
References	52
A Appendix: DML	56
B Appendix: OOD Detection	57

List of Figures

2.1	<i>Siamese Network for Face Recognition</i>	5
2.2	<i>Distance relationship for a Siamese Network. [2]</i>	6
2.3	<i>Violations of the three similarities for negative pairs. [3]</i>	8
3.1	<i>Language-guidance for better semantic visual alignment. [4]</i>	10
3.2	<i>Language guidance: (A) Expert Language Guidance (ELG) and (B) Pseudolabel Language Guidance (PLG) [4]</i>	11
3.3	<i>State-of-the-art Results of Language-guidance. [4]</i>	14
3.4	<i>Experiment: Our results (recall score). [5]</i>	15
3.5	<i>Experiment: Our results (f1 score). [5]</i>	15
4.1	<i>Hint Learning + Knowledge Distillation method for Image Classification; Red arrows represent back-propagation.</i>	17
4.2	<i>Hint Learning Process of our Model</i>	19
5.1	<i>Hint Learning Results (recall score).</i>	20
5.2	<i>Hint Learning Results (f1 score).</i>	21
5.3	<i>Hint Learning Results (Training Loss).</i>	21
7.1	<i>SelectiveNet Basic Diagram [6]</i>	30
7.2	<i>ROC Curve</i>	32
7.3	<i>Precision-Recall Curve</i>	32
8.1	<i>Zero-shot OOD detection framework using MCM and CLIP model. [7]</i> . .	36
8.2	<i>MCM scores for Pet37 as ID Dataset with various OOD Datasets</i>	38
8.3	<i>FPR95, AUROC and AUPR scores for Pet37 as ID Dataset with various OOD Datasets</i>	39
8.4	<i>MCM scores for bird200 as ID Dataset with various OOD Datasets</i> . . .	39
8.5	<i>FPR95, AUROC and AUPR scores for bird200 as ID Dataset with various OOD Datasets</i>	40
9.1	<i>CoOp Architecture [8]</i>	43
9.2	<i>CoCoOp Architecture [9]</i>	44
10.1	<i>Result of CLIP+CoOp method tested on 11 datasets. [8]</i>	47
10.2	<i>Training Loss for CLIP+MCM+CoOp model</i>	47

10.3	<i>Training Accuracy for CLIP+MCM+CoOp model</i>	48
10.4	<i>Visualizing 3 nearest words for the 15 learned context tokens</i>	49
10.5	<i>Visualizing 3 nearest words for the 15 learned context tokens</i>	49
10.6	<i>OOD Detection Results of our model (MCM + Prompt Learning) for various OOD datasets</i>	50

List of Tables

8.1	Results of MCM OOD detection using CLIP on different datasets.	40
8.2	Comparison of MCM with other OOD detection methods. [7]	41

Part I

Deep Metric Learning (DML)

Chapter 1

Introduction: Multi-Modal DML

1.1 Motivation

Vision-based Similarity Learning with deep neural networks, i.e. Deep Metric Learning, involves using deep neural networks to learn similarities or distances between data points, which plays a critical role in various computer vision and machine learning applications.

Deep Metric Learning has many applications such as image retrieval or image search (eg. Google’s reverse image search) [10, 11, 12, 13, 14], clustering [15, 16], fine-grained retrieval [17], face recognition and Kinship Verification [18, 19], zero-shot learning [12], contrastive supervised [20] and unsupervised [21] representation learning [22], etc.

Today, everyone has easy access to a large number of digital images. Deep learning is becoming increasingly popular as a result of the readily available huge datasets. Image retrieval and other similar tasks are becoming more crucial for a variety of applications. To meet the demands of these applications, there is a need for fast, accurate, and scalable deep metric learning approaches.

1.2 Background

Deep Metric Learning (DML) is a well-established framework for learning visual similarities. It involves training models to learn metric representations or embedding spaces, where the distance between samples reflects their semantic similarity, such as Euclidean or cosine distance.

In Vision-based Similarity Learning, it is often necessary to generalize the learned task beyond the training classes. This requires capturing high-level semantic context that goes beyond the relationships observed during training. However, most DML training methods rely solely on the class labels provided in the datasets. As a result, every class is treated equally, and the arrangement of classes in the embedding space is determined

solely by the ranking tasks based on the class labels. Consequently, the high-level semantic relationships between different classes are not considered.

To address this issue, one approach is to leverage the vast corpora of large-scale pre-trained natural language models that are readily available. These models provide contextual information for class labels independent of the specific task, enabling DML models to learn visually consistent representation spaces that align more closely with semantic relationships.

Language Guidance is a technique used to incorporate pretrained language models into visual similarity learning. Two methods commonly employed for Language Guidance are Expert Language Guidance (ELG) and Pseudolabel Language Guidance (PLG) [4]. ELG requires expert knowledge (i.e. pre-labeled class names), while the class names in PLG are obtained with the help of ImageNet pretrained backbone. The details of Language Guidance, including ELG and PLG, will be discussed further in the following section.

1.3 Contribution

In order to further enhance the performance of the model, we propose the utilization of a technique called Hint Learning in conjunction with Language Guidance. Our approach involves extracting features from intermediate layers of the model and calculating the loss to train the model up to that particular layer. Subsequently, the entire model is trained using Language Guidance. We conduct a comprehensive analysis of the results obtained from our technique, which is discussed in detail in the fourth chapter.

1.4 Organization of the Report

The introduction chapter provides a summary of the motivation for the chosen topic, the background and our contribution. The second chapter covers Deep Metric Learning (DML) i.e. Vision-based Similarity Learning. The third chapter goes into great detail about Language Guidance. The fourth chapter discusses the Hint Learning approach that we suggested. The results, conclusions, and potential future changes to the model are covered in the fifth chapter.

Chapter 2

Deep Metric Learning

2.1 Description of Deep Metric Learning

Deep Metric Learning is a powerful subfield of machine learning that focuses on learning meaningful and discriminative representations for data points in a high-dimensional space. Unlike traditional methods that aim to classify or regress data directly, Deep Metric Learning concentrates on understanding the underlying relationships between samples. The primary objective is to bring similar data points closer together while pushing dissimilar ones apart in the learned feature space. This process enables the model to capture intrinsic similarities among instances and create an embedding space that encodes semantically meaningful information. By utilizing large-scale labeled or even weakly labeled datasets, Deep Metric Learning algorithms can produce compact and expressive representations.

The versatility and effectiveness of Deep Metric Learning have led to significant advancements in various domains and offer promising prospects for enhancing the performance of real-world applications. In this project, we are using Vision-Based DML methods which learn the similarity/relation between images, enabling us to create a robust and efficient system for image retrieval, content-based image clustering and other tasks.

2.2 Applications of Deep Metric Learning

Deep Metric Learning has found wide-ranging applications across various domains due to its ability to learn informative and discriminative embeddings for data. In computer vision, it has revolutionized image retrieval, enabling systems to efficiently search and retrieve visually similar images from vast databases. Face recognition systems have also greatly benefited from Deep Metric Learning, facilitating accurate identification and verification of individuals even in challenging conditions (One approach for Face recognition is using Siamese Network, as seen in Figure 2.1). Moreover, it plays a crucial role in person re-identification, allowing surveillance systems to track individuals across different cameras and locations.

Beyond computer vision, Deep Metric Learning has been employed in natural language processing for tasks such as sentence similarity, document clustering, and information retrieval, where it can capture semantic relationships between textual data. Additionally, it has shown promise in recommendation systems, personalized medicine, and anomaly detection, further demonstrating its versatility and potential impact on diverse real-world applications. As research in this field continues to evolve, the scope of Deep Metric Learning is expected to expand, leading to even more groundbreaking solutions in the future.

2.3 Methods used in Deep Metric Learning

2.3.1 Contrastive Loss (Siamese Networks)

Siamese Network is a popular architecture used for Deep Metric Learning. Siamese Networks are mainly used in combination with a Contrastive Loss function, also called Pairwise ranking Loss. Siamese Networks consist of twin neural networks that share the same weights and learn to map input pairs into a common feature space (hence computing the same function). The objective of contrastive loss is to encourage similar samples to be closer together and dissimilar samples to be farther apart in the learned feature space. When training the network, it takes pairs of samples and computes their embeddings. The contrastive loss function then penalizes the distance between embeddings of similar samples while encouraging a margin of separation for embeddings of dissimilar ones. This way, the Siamese Networks are capable of capturing the underlying similarity information effectively, producing discriminative embeddings that facilitate various DML tasks like image retrieval and clustering. Contrastive loss has demonstrated remarkable performance and remains a fundamental component in many state-of-the-art Deep Metric Learning approaches, continuing to drive advancements in this rapidly evolving field.

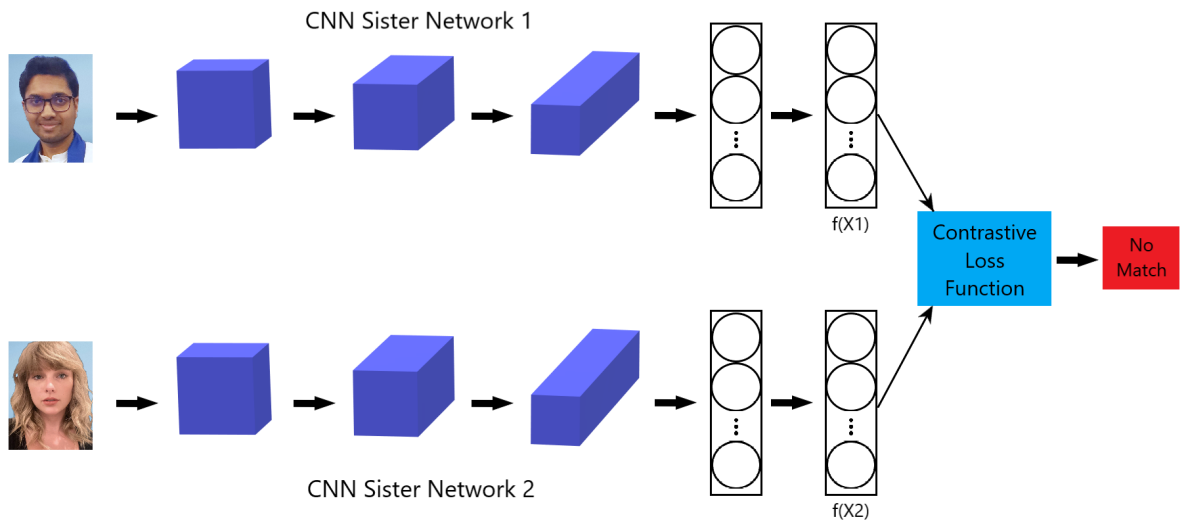


Figure 2.1: *Siamese Network for Face Recognition*

Siamese Networks can be used for tasks such as Face Recognition (See Figure 2.1), where the network learns to map facial images into a feature space, enabling accurate identification and verification of individuals even in challenging conditions.

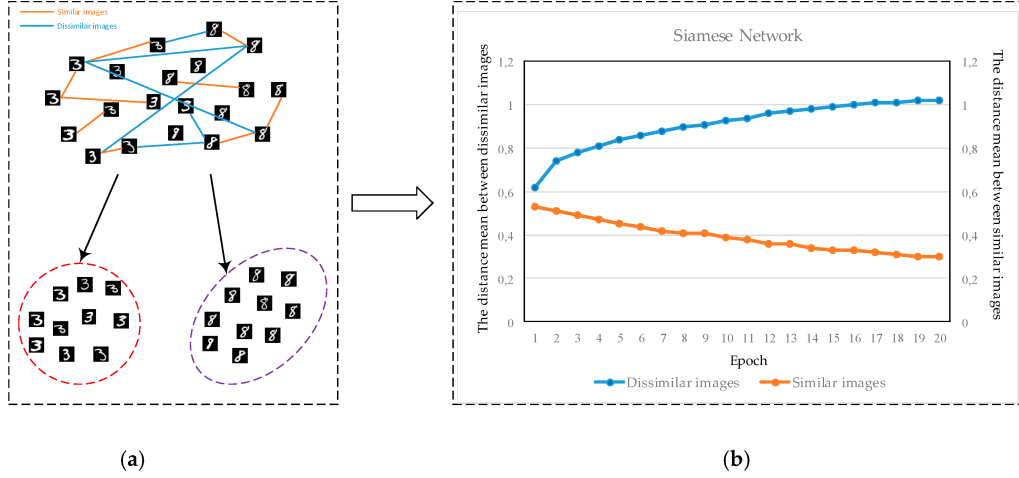


Figure 2.2: *Distance relationship for a Siamese Network.* [2]

As the Siamese Networks undergo more training epochs, the distance between dissimilar images gradually increases, while the distance between similar images decreases, as depicted in Figure 2.2 (b). This phenomenon is reflected in the visualization of the embedding space, where images with similar features cluster together, as illustrated in Figure 2.2 (a).

Let's take a look at the Contrastive Loss Function:

$$\mathcal{L}_{\text{Contrastive}} = (Y) \|f(X_1) - f(X_2)\|_2 + (1 - Y) \{\max(0, m - \|f(X_1) - f(X_2)\|_2)\} \quad (2.1)$$

Here, $\|f(X_1) - f(X_2)\|_2$ is the distance (L2 norm) for a pair of input samples. Y is 1 if the input pairs are positive pairs and 0 if they are negative. m is the margin.

The margin (m) plays a crucial role in the Contrastive Loss Function. Only negative pairs with an estimated distance less than m contribute to the loss, disregarding pairs that exceed this limit. As a result, the network can concentrate more on embedding challenging data points effectively, as it no longer considers the distances of negative pairs beyond the specified margin. This mechanism allows the model to focus on learning meaningful and discriminative representations for difficult instances, enhancing the overall performance.

2.3.2 Triplet Loss

The Triplet network is a symmetric neural architecture with three identical subnetworks sharing the same parameters. It operates on sets of three images: the Anchor image (baseline), the Positive image (belonging to the same class as the anchor), and the Negative image (not belonging to the anchor class). The objective of the triplet loss is to minimize the Euclidean distance between the anchor and positive image while maximizing the distance between the anchor and negative image. In essence, the triplet loss ensures that the anchor image is closer to all positive images and far from the negative images, thus promoting effective feature embeddings for similarity-based tasks like image retrieval and face recognition.

Let's take a look at the Triplet Loss Function:

$$\mathcal{L}_{\text{Triplet}} = \max(0, \|f(X) - f(X^p)\|_2 - \|f(X) - f(X^n)\|_2 + \alpha) \quad (2.2)$$

Here, X = anchor image, X^p = positive image, X^n = negative image, α = margin.

Triplet Loss and Contrastive Loss suffer from a common issue, which is the need for a time-consuming and performance-sensitive pair/triplet mining procedure during model training.

2.3.3 Multi Similarity Loss

Multi Similarity Loss (MSL) is an innovative approach in Deep Metric Learning (DML) that addresses some of the challenges posed by traditional loss functions like triplet loss and contrastive loss. MSL aims to better utilize the information present in the data by considering both positive and negative pairs simultaneously. Unlike triplet loss that focuses on only one positive and one negative sample for each anchor, MSL incorporates multiple positive and negative samples into the loss computation. By doing so, MSL enhances the discriminative power of the learned embeddings and provides a more comprehensive understanding of the data's underlying structure. This approach also eliminates the need for time-consuming pair/triplet mining procedures, simplifying the training process and improving computational efficiency. Through its ability to exploit the richness of the data and generate more effective embeddings, Multi Similarity Loss has shown promising results in various DML tasks, such as image retrieval, face recognition, and clustering.

Let's begin by examining the three types of similarities that significantly influence sample selection and weighting in deep metric learning. Following that, we delve into the concept of multi-similarity loss, which takes into account all three similarities through iterative sample mining and weighting.

The three similarities for pair-based methods are described as follows:

S: Self-similarity: A self-similarity is computed from the pair itself and is considered the most important. A negative pair with a larger cosine similarity means that it is harder to distinguish two paired samples from different classes. Such pairs are referred to as hard negative pairs, which are more informative and meaningful to learning discriminatory features. As shown in case-1 of Figure 2.3, the weights of three negative pairs are increased, when the negative samples come closer

N: Negative relative similarity: It is computed by considering the relationship from neighboring negative pairs. As described in case-2 of Figure 2.3, even when the self-similarity remains unchanged, the relative similarity of a pair can decrease as its neighboring negative samples move closer, reducing the overall relative similarity.

P: Positive relative similarity: Similarly, the relative similarity also considers the relationship from other positive pairs, with the same anchor. As shown in case-3 of Figure 2.3, if positive samples become closer to the anchor, the relative similarity of the current pair decreases, leading to a corresponding reduction in the pair’s weight.

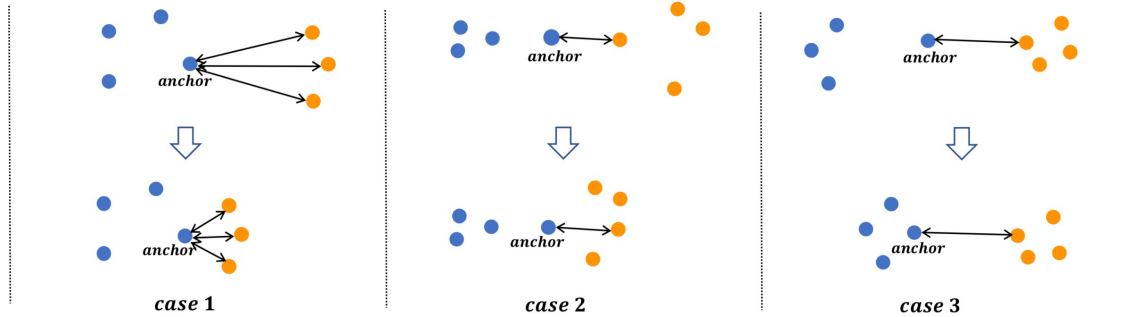


Figure 2.3: *Violations of the three similarities for negative pairs. [3]*

Case-1: The negative pairs’ cosine similarities increase as they come closer to the anchor;

Case-2: Relative similarity compared with negatives decreases as the other negatives’ cosine similarities increase;

Case-3: Relative similarity decreases as relevant positive pairs become closer.

By jointly considering all three similarities through iterative sample mining and weighting, the Multi-Similarity (MS) loss achieves more comprehensive and discriminative feature learning in deep metric learning tasks: (i) informative pairs are first sampled by measuring Similarity-P, and then (ii) the selected pairs are further weighted using Similarity-S and Similarity-N jointly.

Pair mining: For pair mining, Similarity-P is computed to select informative pairs. Similarity-P measures the relative similarity between negative-positive pairs that share the same anchor. To achieve this, we compare a negative pair with the hardest positive pair (i.e., the one with the lowest similarity). Conversely, for sampling positive pairs, we compare them with negative pairs that exhibit the highest similarity.

Assume x_i is an anchor, a negative pair (x_i, x_j) is selected if S_{ij} satisfies the condition:

$$S_{ij}^- > \min_{\mathbf{y}_k = \mathbf{y}_i} S_{ik} - \epsilon \quad (2.3)$$

If (x_i, x_j) is a positive pair, the condition is:

$$S_{ij}^+ < \max_{\mathbf{y}_k \neq \mathbf{y}_i} S_{ik} + \epsilon \quad (2.4)$$

Where ϵ is a given margin, and S_{ij} is the similarity of two samples.

Pair weighing: Pair mining can roughly select informative pairs and eliminate less informative ones. To achieve more accurate weighting of the selected pairs, Pair Weighting is implemented by using a soft weighting scheme that takes into account both Similarity-S and Similarity-N.

Specifically, given a selected negative pair $x_i, x_j \in N_i$, its weight w_{ij}^- can be computed as:

$$\begin{aligned} w_{ij}^- &= \frac{1}{e^{\beta(\lambda - S_{ij})} + \sum_{k \in \mathcal{N}_i} e^{\beta(S_{ik} - S_{ij})}} \\ &= \frac{e^{\beta(S_{ij} - \lambda)}}{1 + \sum_{k \in \mathcal{N}_i} e^{\beta(S_{ik} - \lambda)}} \end{aligned} \quad (2.5)$$

while the weight w_{ij}^+ of a positive pair $x_i, x_j \in P_i$ can be computed as:

$$w_{ij}^+ = \frac{1}{e^{-\alpha(\lambda - S_{ij})} + \sum_{k \in \mathcal{P}_i} e^{-\alpha(S_{ik} - S_{ij})}}, \quad (2.6)$$

where α, β, λ are fixed hyper-parameters.

The multi-similarity (MS) loss, a pair-based loss function, is computed by integrating pair mining and weighting scheme into a single framework. It's partial derivative with respect to S_{ij} is the weight defined.

$$\begin{aligned} \mathcal{L}_{MS} &= \frac{1}{m} \sum_{i=1}^m \left\{ \frac{1}{\alpha} \log \left[1 + \sum_{k \in \mathcal{P}_i} e^{-\alpha(S_{ik} - \lambda)} \right] \right. \\ &\quad \left. + \frac{1}{\beta} \log \left[1 + \sum_{k \in \mathcal{N}_i} e^{\beta(S_{ik} - \lambda)} \right] \right\}. \end{aligned} \quad (2.7)$$

Chapter 3

Language Guidance

3.1 Introduction to Language Guidance

As mentioned in the Introduction, many Deep Metric Learning (DML) approaches tend to overlook the higher-level semantic relations among actual classes. Consequently, the learned embedding spaces may lack essential semantic context and misrepresent the relationships between classes, leading to reduced generalizability of the learned metric space. To address this limitation, a method called Language Guidance is employed for visual similarity learning. This approach leverages language guidance objectives to enhance DML by incorporating higher-level semantic information. Model-agnostic language guidance proves to be highly beneficial, yielding competitive and state-of-the-art results across various benchmarks. There are two main approaches to Language Guidance: Expert Language Guidance (ELG) and Pseudolabel Language Guidance (PLG), which will be further explored in subsequent sections.

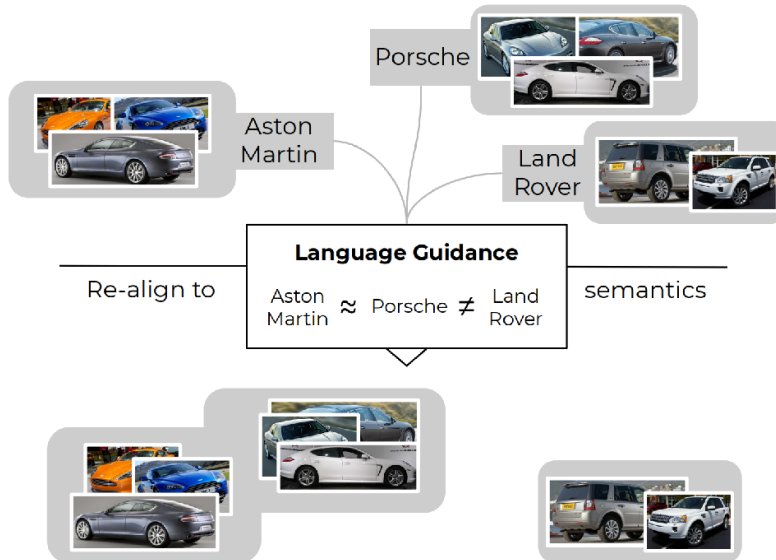


Figure 3.1: *Language-guidance for better semantic visual alignment.* [4]

3.2 Language Guidance Model Architecture

The Language Guidance Model Architecture extends the default DML pipeline for Visual Similarity Learning by incorporating a pretrained language model. This language model can be fed with either expert class names or top-k ImageNet pseudolabels, eliminating the need for additional expert supervision. As a result, language similarity matrices are obtained, which play a pivotal role in guiding the structuring of the fine-grained visual similarity space generated by our model. This is achieved through a distillation-based matching process, aligning the language and image similarity matrices.

During the training process (see Figure 3.2), the DML loss is computed as usual by projecting the images into a metric space. Concurrently, the class labels, whether hand-labeled or generated using an ImageNet pretrained model, are processed by a language encoder to obtain the text projections. The Language Guidance loss is then calculated based on the relationship between the text and image projections. By combining the Language Guidance and DML losses, a unified loss function is created, which is back-propagated through the image model during training. Further details on the training process will be elaborated in subsequent sections of this report.

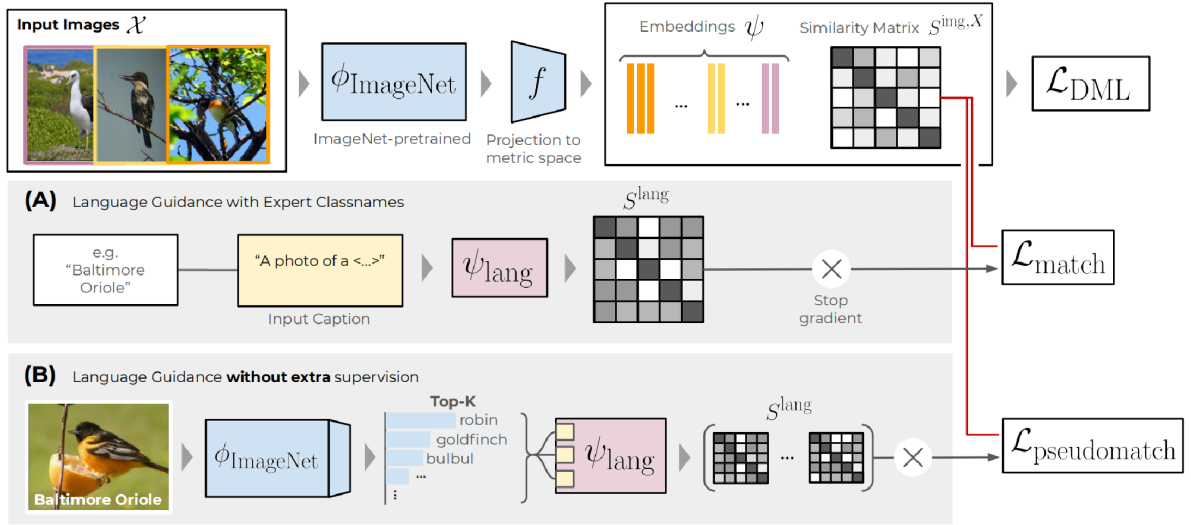


Figure 3.2: *Language guidance: (A) Expert Language Guidance (ELG) and (B) Pseudolabel Language Guidance (PLG)* [4]

In the Figure 3.2 , part (A) is Language Guidance with Expert Classnames, which requires pre-labeled expert classnames. And part (B) is Language Guidance without expert classnames - in this method, the labels (class names) for images are obtained using ImageNet pretrained model (or any other suitable CNN model).

3.2.1 Expert Language Guidance (ELG)

The Expert Language Guidance (ELG) approach utilizes expert class label names, i.e. hand engineered class labels, as shown in the Figure 3.2A.

To incorporate language semantics into visual similarity learning via ELG, we leverage large pretrained language models like CLIP [23], BERT [24], RoBERTa [25], etc., which are capable of mapping input text (class labels) to their corresponding embedding space. To generate input sequences for the natural language model, we use a prompt like "A photo of a $a < y_i$ ".

Let S^{lang} represent the batch-wise cosine similarity matrix for all language representations in the minibatch, and let S^{img} be the corresponding visual similarity matrix. The language distillation loss is then defined as:

$$\mathcal{L}_{\text{match}}(S^{\text{img}}, S^{\text{lang}}) = \frac{1}{|\mathcal{B}|} \sum_i \sigma(S_i^{\text{img}, X}) \log \left(\frac{\sigma(S_i^{\text{img}, X})}{\sigma(S_i^{\text{lang}} + \gamma_{\text{lang}})} \right) \quad (3.1)$$

The loss in Equation 3.1 represents a KL-Divergence matching between row-wise image and language similarities. In this equation, σ denotes the row-wise softmax function with a shift γ_{lang} , which is used to address the fidelity of the distribution matching between the image and language similarity distributions. [4]

S^{lang} does not resolve similarities for samples within a class unlike S_{img} (for $x_i \neq x_j$ and $y_i = y_j$ we have $\psi_i \neq \psi_j$ and thus $S_{i,j}^{img} < 1$, but, since the classes are the same, $S^{lang} = 1$). To preserve intraclass resolution during the distillation, S^{img} is modified as $S_{i,j}^{img, X}$:

$$S_{i,j}^{img, X} = I_{y_i=y_j} [1 + \gamma_{\text{lang}}] + I_{y_i \neq y_j} [S_{i,j}^{img}] \quad (3.2)$$

$S_{i,j}^{img, X}$ performs a masking operation on the respective row entries of the original visual similarity matrix S^{img} . This masking is applied to rows where the classes are the same as the respective anchor class, along with the respective language offset. The purpose of this masking is to ignore matching within-class similarities for datasets where there is no sample-specific natural language description available. By masking out these entries, we ensure that the model does not utilize language information for samples within the same class, thus preventing any potential bias introduced by the absence of specific natural language descriptions.

$$\mathcal{L}_{ELG} = \mathcal{L}_{\text{DML}} + \omega \cdot \mathcal{L}_{\text{match}} \quad (3.3)$$

The overall ELG Loss is obtained by computing a weighted sum of DML loss and $\mathcal{L}_{\text{match}}$ with ω (see Equation 3.3)

Note: For both the ELG and PLG, Backpropagation only happens through the vision model during the training, since the language model only offers the language target that the image representation space should be oriented towards.

3.2.2 Pseudolabel Language Guidance (PLG)

Pseudolabel Language Guidance (PLG) offers an effective solution to leverage language context in visual similarity learning without the need for expert class labelling, which is required in the ELG approach.

In PLG, we take advantage of the ImageNet pretrained backbone commonly used in deep metric learning (DML) pipelines. Instead of relying solely on pretrained features, we utilize the classifier of the pretrained model to generate ImageNet pseudolabels. This process involves running both the pretrained backbone and classifier on all images within a class to obtain softmax outputs corresponding to all ImageNet classes. For each sample, these softmax outputs are averaged for each class, and the top-k ImageNet pseudo-class names are selected to represent the class. (see Figure 3.2 (B))

While the pseudo class labels obtained from the classifier may not be as precise as expert labels, they still provide a valuable connection between the recognized object in the target image and the likely true label in most cases. To enhance the semantic context, the language network is applied to these pseudolabels, providing a rough but informative language representation. This process is repeated for all k of the top-k ImageNet labels corresponding to each class, resulting in a more accurate and detailed language resolution. By considering multiple top-k labels rather than just a single pseudolabel, we obtain a substantially improved semantic description. The language similarity matrices for all the top-k labels are computed and merged by taking the average, thereby capturing a more comprehensive language representation. This merged language similarity matrix is then used in the training process by calculating the training loss (See equations 3.4 and 3.5).

$$\mathcal{L}_{\text{pseudomatch}}^k = \mathcal{L}_{\text{match}} \left(S^{\text{img}}, \frac{1}{k} \sum_j^k S^{\text{pseudolang}, j} \right) \quad (3.4)$$

In eq 3.4, instead of using the masked image similarities $S^{\text{img}, X}$, the unmasked variant S^{img} can be utilized to account for the semantic differences between images of the same class. This is possible because the pseudolabels can be extracted at the sample level, providing a more fine-grained representation, rather than just at the class level. By incorporating the unmasked image similarities, the model can better capture and distinguish variations within the same class, leading to improved performance in capturing intra-class resolution during the distillation process.

However, in practice there were no improvements on the evaluated benchmarks. [4]

$$\mathcal{L}_{PLG} = \mathcal{L}_{DML} + \omega \cdot \mathcal{L}_{pseudomatch}^k \quad (3.5)$$

The overall loss for PLG is obtained similar to the ELG by computing a sum of DML loss and pseudomatch loss for top-k labels $\mathcal{L}_{pseudomatch}^k$, weighted by ω . (Refer to the Equation 3.5)

3.3 Results of Language Guidance

3.3.1 Comparison

BENCHMARKS →	CUB200			CARS196			SOP		
METHODS ↓	R@1	R@2	NMI	R@1	R@2	NMI	R@1	R@10	NMI
ResNet50, 128 dim.									
Margin	63.6	74.4	69.0	79.6	86.5	69.1	72.7	86.2	90.7
Div&Conq	65.9	76.6	69.6	84.6	90.7	70.3	75.9	88.4	90.2
MIC	66.1	76.8	69.7	82.6	89.1	68.4	77.2	89.4	90.0
PADS	67.3	78.0	69.9	83.5	89.7	68.8	76.5	89.0	89.9
S2SD	68.9 ± 0.3	79.0 ± 0.3	72.1 ± 0.4	87.6 ± 0.2	92.7 ± 0.2	72.3 ± 0.2	80.2 ± 0.2	91.5 ± 0.1	90.9 ± 0.1
Multisimilarity+PLG	67.8 ± 0.2	78.2 ± 0.2	70.1 ± 0.1	86.0 ± 0.3	91.4 ± 0.1	72.4 ± 0.2	77.9 ± 0.1	89.9 ± 0.2	90.2 ± 0.2
S2SD+PLG	71.1 ± 0.1	80.6 ± 0.2	73.0 ± 0.2	89.1 ± 0.2	93.8 ± 0.2	73.1 ± 0.3	80.6 ± 0.1	91.8 ± 0.2	90.9 ± 0.1
ResNet50, 512 dim.									
EPSHN	64.9	75.3	-	82.7	89.3	-	78.3	90.7	-
NormSoft	61.3	73.9	-	84.2	90.4	-	78.2	90.6	-
DiVA	69.2	79.3	71.4	87.6	92.9	72.2	79.6	91.2	90.6
DCML-MDW	68.4	77.9	71.8	85.2	91.8	73.9	79.8	90.8	90.8
IB-DML	70.3	80.3	74.0	88.1	93.3	74.8	81.4	91.3	92.6
Multisimilarity+PLG	69.6 ± 0.4	79.5 ± 0.2	70.7 ± 0.1	87.1 ± 0.2	92.3 ± 0.3	73.0 ± 0.2	79.0 ± 0.1	91.0 ± 0.1	90.0 ± 0.1
S2SD+PLG	71.4 ± 0.3	81.1 ± 0.2	73.5 ± 0.3	90.2 ± 0.3	94.4 ± 0.2	72.4 ± 0.3	81.3 ± 0.2	92.3 ± 0.2	91.1 ± 0.2
Inception-BN, 512 dim.									
Group	65.5	77.0	69.0	85.6	91.2	72.7	75.1	87.5	90.8
Multisimilarity	65.7	77.0	-	84.1	90.4	-	78.2	90.5	-
DR-MS	66.1	77.0	-	85.0	90.5	-	-	-	-
ProxyGML	66.6	77.6	69.8	85.5	91.8	72.4	78.0	90.6	90.2
ProxyAnchor	68.4	79.2	-	86.8	91.6	-	79.1	90.8	-
Multisimilarity+PLG	69.2 ± 0.2	79.7 ± 0.1	70.6 ± 0.3	86.2 ± 0.2	91.5 ± 0.3	70.8 ± 0.3	78.6 ± 0.2	90.7 ± 0.1	90.0 ± 0.2
S2SD+PLG	70.4 ± 0.2	80.5 ± 0.2	71.9 ± 0.3	88.1 ± 0.3	92.9 ± 0.1	71.4 ± 0.3	79.4 ± 0.1	91.2 ± 0.1	90.4 ± 0.2

Figure 3.3: *State-of-the-art Results of Language-guidance.* [4]

The results of the experiments conducted in the research paper titled "Integrating Language Guidance into Vision-based Deep Metric Learning" [4] are presented in Figure 3.3. The Table clearly demonstrates that the Language Guidance methods exhibit superior performance compared to all previous approaches. The Language Guidance techniques showcase a substantial margin of improvement, outperforming other methods significantly. The experimental findings provide compelling evidence for the benefits of using language models to guide the learning process.

3.3.2 Our Results and Conclusion

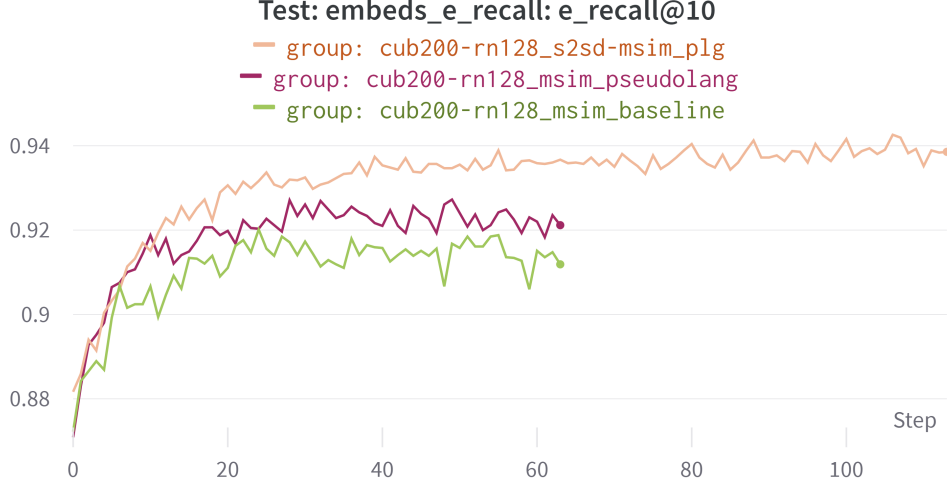


Figure 3.4: *Experiment: Our results (recall score).* [5]

Additionally, we tested the model's performance by utilising three different techniques to train it on the cub200 dataset (The results can be seen in the Figures 3.4 & 3.5). Methods we used are:

- Baseline DML with Multi Similarity Loss; Without Language Guidance (Green Color)
- PLG with Multi Similarity Loss (Ruby Color)
- PLG with S2SD method (Orange Color)

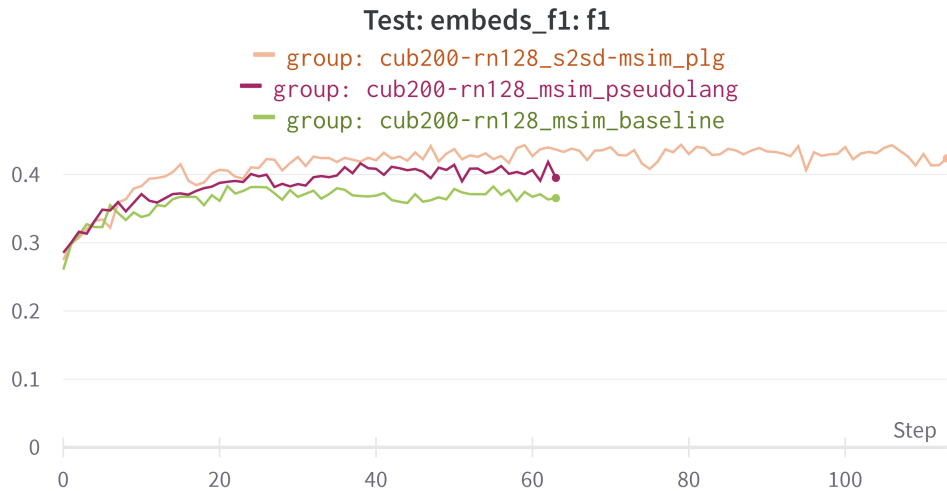


Figure 3.5: *Experiment: Our results (f1 score).* [5]

In Figure 3.4, we observe that the Baseline model exhibits the lowest performance compared to other methods. However, incorporating the Pseudolabel Language Guidance (PLG) into the baseline model leads to a notable improvement in performance. Moreover, applying the S2SD [26] (Simultaneous Similarity-based Self-Distillation) approach on top of the PLG-enhanced model results in a substantial boost in performance. The performance of these three methods can be further analyzed in Figure 3.4, where we use the Recall metric, and in Figure 3.5, where we use the f1 score as the evaluation criteria.

Our experimental results align closely with the findings reported by the authors of the referenced paper [4]. This indicates a consistency between our results and the results presented in the paper, it further strengthens the validity of our approach and validates the claims made in the research paper.

Our experimental outcomes reinforce the potential of incorporating Language Guidance into vision-based deep metric learning, showing its effectiveness in capturing richer semantic relationships between classes and enhancing the overall performance of the model.

Chapter 4

Hint Learning

4.1 Introduction to Hint Learning

Hint Learning is a valuable technique that enhances the performance of a student network by leveraging the intermediate representations of a teacher network as guiding hints during the training process. These hints are the outputs of the teacher's hidden layers, which assist in directing the student's learning. Specifically, the layer in the teacher network whose output serves as the guidance for the student is termed the "Hint" layer, while the corresponding layer in the student network that learns from the hint is referred to as the "guided" layer. The objective function is designed to minimize the loss between the outputs of the Hint and Guided layers. This approach ensures that the guided layer can predict the output of the hint layer effectively. Notably, hint learning serves as a form of regularization, but it is essential to select the appropriate pair of hint and guided layers to prevent over-regularization of the student network. By incorporating hint learning, we empower the student network to benefit from the teacher's expertise, leading to improved performance and more efficient learning.

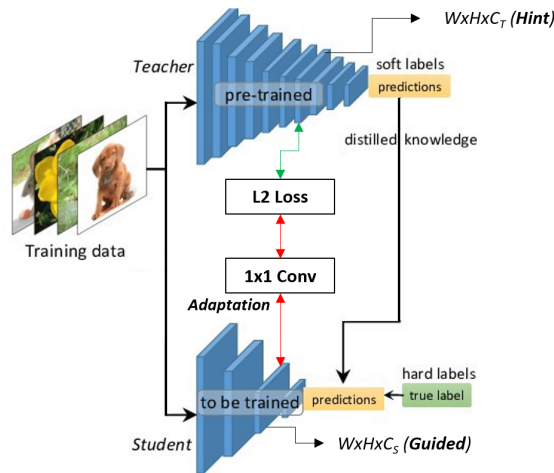


Figure 4.1: *Hint Learning + Knowledge Distillation method for Image Classification; Red arrows represent back-propagation.*

As the teacher network typically has a wider architecture than the student network, the chosen hint layer may have more outputs than the guided layer. To address this disparity in output dimensions, a regressor is introduced, which acts as a bridge between the guided layer and the hint layer. The regressor’s role is to transform the outputs of the guided layer to match the size of the hint layer. Subsequently, during training, both the student parameters from the first layer up to the guided layer and the parameters of the regressor are optimized by minimizing the loss function. This approach ensures that the student network can effectively learn from the teacher’s hints and adapt its guided layer to match the intermediate representations of the wider hint layer. (Refer to the Figure 4.1)

$$\mathcal{L}_{Hint}(\mathbf{W}_{Guided}, \mathbf{W}_r) = \frac{1}{2} \|u_h(\mathbf{x}; \mathbf{W}_{Hint}) - r(v_g(\mathbf{x}; \mathbf{W}_{Guided}); \mathbf{W}_r)\|^2, \quad (4.1)$$

The hint learning loss, as represented by Equation 4.1, involves the teacher and student deep nested functions, denoted as u_h and v_g respectively, up to their corresponding hint and guided layers. These functions have parameters W_{Hint} and W_{Guided} , respectively. Additionally, a regressor function, represented by r and parameterized by W_r , is employed on top of the guided layer. It is crucial to ensure that the outputs of u_h and r are comparable, which means that they must share the same non-linearity. This requirement ensures that the knowledge transfer between the teacher’s hint layer and the student’s guided layer is effective and preserves the consistency of the representations across the two layers. By adhering to this condition, we enable the guided layer to successfully learn from the teacher’s hints and align its outputs with the intermediate representations from the wider hint layer.

4.2 Method

The training process of our model (Hint Learning + Language Guidance) consists of two distinct phases:

In the first phase (refer to Figure 4.2), we train the vision model (student) using the hint learning loss. For this purpose, a specific layer from the vision model, responsible for projecting the image onto a latent space, is selected as the guided layer. Simultaneously, another layer from the language encoder model (teacher) is chosen as the hint layer. The middle layer features are extracted from both the vision and language models, with the extracted features from the vision model having a shape of $(BatchSize, NumberOfChannels, Height, Width)$, and the language model features having a shape of $(BatchSize, 512)$. To align the dimensions of the vision and language features, the vision features are flattened and passed through a fully connected layer, acting

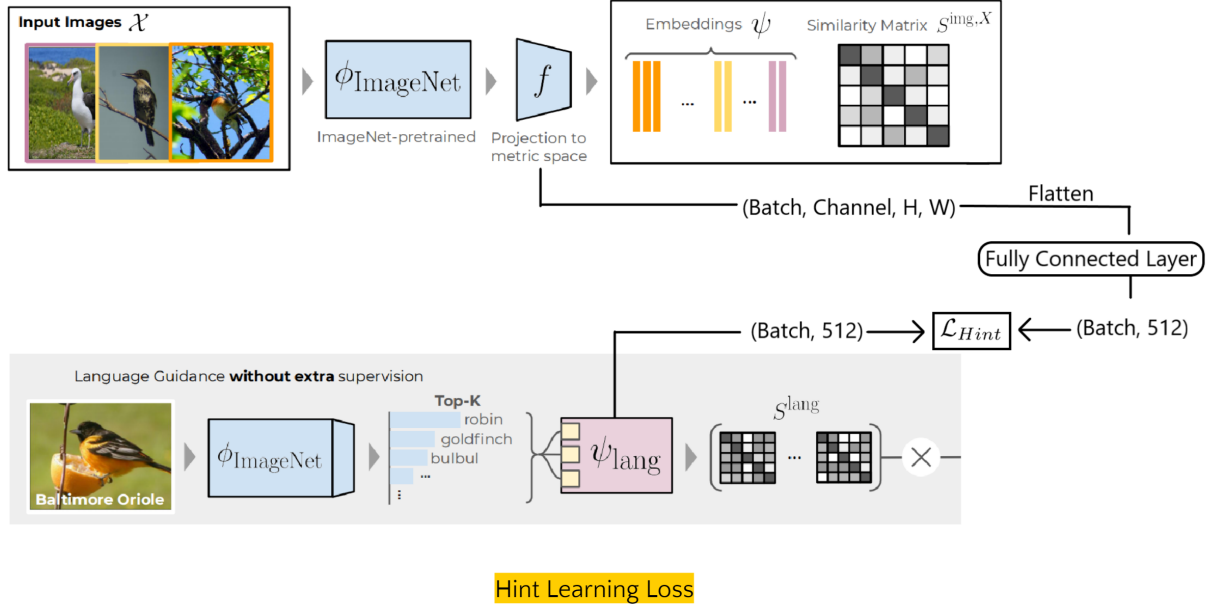


Figure 4.2: *Hint Learning Process of our Model*

as a regressor, resulting in a final output of size $(BatchSize, 512)$. This output is then compared to the language features using the Mean Squared Error (MSE) loss function, as depicted in Equation 4.1. Importantly, during this phase, only the vision model is back-propagated for training.

In the second phase, we proceed with training the model using the Language Guidance method, preferably the Parametric Language Guidance (PLG) method discussed in the previous chapter. This second phase complements the hint learning process and further refines the model’s performance by incorporating language guidance in the learning process. By combining these two phases, our model benefits from both the hints provided by the teacher’s language encoder layer and the language guidance method, resulting in an effective and comprehensive training approach that improves the model’s performance.

Chapter 5

Results: DML

5.1 Results

We implemented the hint learning method in our code. In this section, we have presented a subset of the results obtained in our study. For a comprehensive overview of all the results, please refer to my Github Repository of the project [5], [1]. The findings of our testing are listed below :

- **Language Guidance (baseline):** PLG with S2SD method (Orange Color)
- **Our Method (middle layers):** PLG+S2SD with Hint learning [hint and guided layers are chosen from the middle layers] (Blue Color)
- **Our Method (last layers):** PLG+S2SD with Hint learning [hint and guided layers are chosen from the last few layers] (Ruby Color)

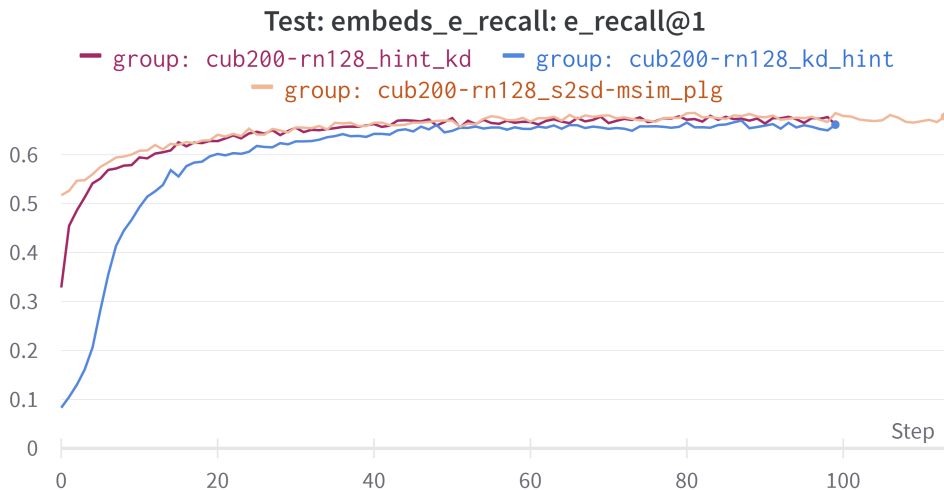


Figure 5.1: *Hint Learning Results (recall score).*

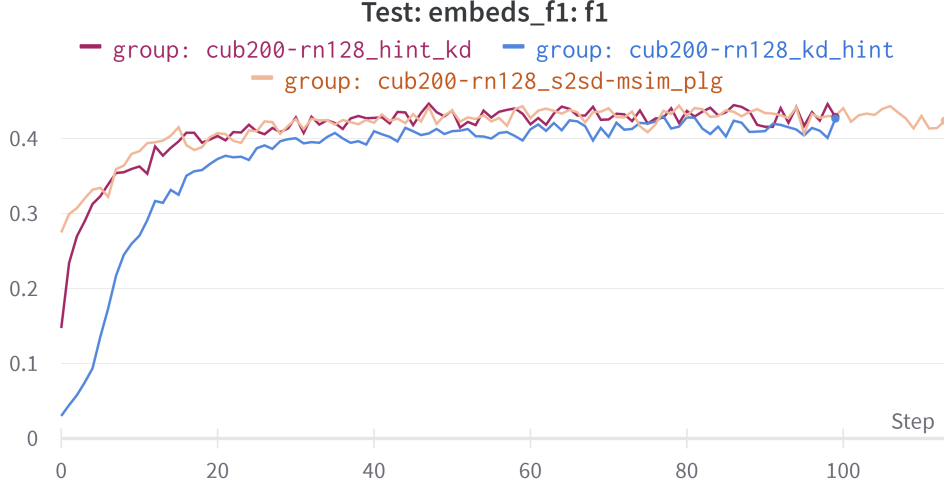


Figure 5.2: *Hint Learning Results (f1 score).*

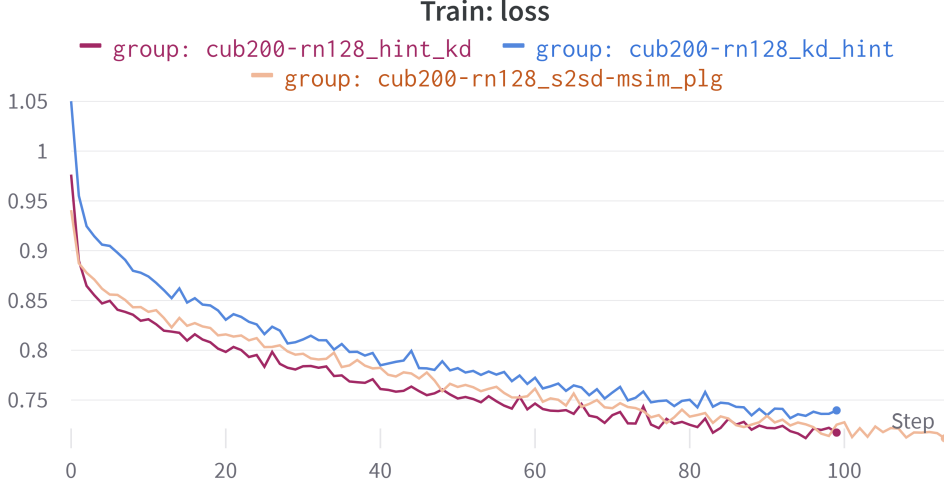


Figure 5.3: *Hint Learning Results (Training Loss).*

5.2 Conclusion

Our approach, Hint Learning + Language Guidance, shares similarities with the Language Guidance Method. However, it does not surpass the performance of the Language Guidance Method. Initially, our method exhibits lower f1 and recall scores compared to the Language Guidance Method. However, as training progresses, these scores gradually converge and become similar. Interestingly, despite the lower f1 and recall scores during the early training phase, our Hint Learning method demonstrates smaller loss values in comparison to the Language Guidance Method (see Figure 5.3). Moreover, even after 80-90 epochs when the recall and f1 scores align, our method continues to maintain a lower loss than the Language Guidance Method. It would be worthwhile to conduct further investigations by training the model for a greater number of epochs. However, due to computational limitations and time constraints, we restricted our training to 100 epochs.

Remarkably, the model that incorporates the hint and guided layers from the last few layers exhibits superior performance compared to the model that utilizes these layers from the middle layers. This can be seen in the Figure 5.1 (recall performance metric) and Figure 5.2 (f1 Score).

This discrepancy in performance can be attributed to the fact that the models we are aligning using Hint Learning belong to different domains, namely Vision and Language. As a result, the initial features may not naturally align since they represent distinct modalities. However, as we move towards the last few layers, the features are expected to converge and project the data into a shared latent space, making alignment more effective and leading to improved performance. By leveraging the information from the last few layers, our model gains a better understanding of the shared representations between Vision and Language domains, ultimately resulting in enhanced alignment and more accurate predictions.

5.3 Future Scope

The future scope for Deep Metric Learning (DML) using our multi-modal method, which incorporates Language Guidance and Hint Learning, opens up several exciting directions for research and application. One potential avenue for exploration is the extension of our approach to more complex and challenging real-world datasets, encompassing diverse domains and modalities. Additionally, further investigations can be conducted to optimize the design and architecture of our multi-modal model, potentially leveraging advanced techniques in language guidance and hint learning to enhance its performance and generalization capabilities.

Moreover, the application of our method can be extended to various computer vision tasks, such as image retrieval, object detection, and semantic segmentation, where deep metric learning plays a pivotal role in improving accuracy and efficiency. Integrating our multi-modal approach into these tasks may yield valuable insights and lead to breakthroughs in the field of computer vision.

The ability to transfer knowledge learned from one domain to another can be advantageous in scenarios where labeled data is limited or unavailable, enhancing the model's ability to perform well in diverse and unseen environments.

Overall, the future scope for multi-modal DML is bright, with numerous possibilities for advancement.

Part II

Out-Of-Distribution (OOD) Detection

Chapter 6

Introduction: Multi-Modal OOD Detection

6.1 Motivation

OOD detection is crucial in machine learning and AI systems because real-world data distributions often change or drift over time. When a model is trained on a specific dataset, it learns patterns and relationships within that dataset to make predictions. However, if the distribution of the incoming data during deployment differs significantly from the training data, the model's predictions may become unreliable or incorrect.

The concept of distribution drift refers to the phenomenon where the statistical properties of the data change over time. This can occur due to various factors, such as changes in user behavior, environmental conditions, or external influences. For example, in a spam email detection system, the characteristics of spam emails may evolve as spammers adopt new techniques, making the model's training data less representative of the current distribution of spam emails. Similarly, in a self-driving car system, the visual appearance of objects on the road can change due to different lighting conditions or new objects that were not present during training.

If an AI system is unaware of these distribution shifts and continues to make predictions based on its original training data, it can lead to prediction errors and potentially harmful consequences. For instance, a speech recognition system trained on data recorded in a quiet environment may fail to accurately transcribe speech in a noisy environment, resulting in misinterpretations and errors in subsequent tasks.

By incorporating OOD detection techniques into AI systems, we can identify samples that come from distributions different from the training data. OOD detection acts as a safeguard, allowing the system to recognize when it encounters inputs that are significantly different from what it has learned. When an OOD sample is detected, the system

can either refuse to make a prediction or take appropriate action to handle the OOD case differently, such as requesting human intervention or deferring to a more suitable model.

By actively monitoring for distribution drift and detecting OOD samples, we can enhance the reliability and safety of AI systems. OOD detection helps prevent the propagation of incorrect predictions and reduces the risk of making decisions based on unreliable information. It enables AI systems to be more robust, adaptive, and better suited for real-world deployment, where the data distribution is likely to evolve over time.

The motivation behind OOD detection with vision-language features arises from the fact that most existing methods for OOD detection focus on a single modality, either vision or language, neglecting the valuable information present in multimodal representations.

6.2 Background

Out-of-Distribution (OOD) detection refers to the task of identifying data samples that differ significantly from the distribution of the training data. It helps machine learning models recognize inputs they were not trained on and prevent making unreliable predictions.

In most cases, OOD detection is typically carried out for a single modality, such as either vision or language. This approach overlooks valuable information that may exist in other modalities. To overcome this limitation, we propose implementing a technique called Maximum Concept Matching (MCM), as introduced in a research paper. MCM aligns visual features with textual concepts, enabling us to harness the combined power of vision and language. By leveraging the synergy between these modalities, we can significantly improve the accuracy and reliability of OOD detection. Additionally, we plan to enhance the MCM method further by incorporating prompt learning. This addition aims to make the technique even more effective in identifying out-of-distribution samples in vision-language applications.

6.3 Contribution

For further improvement of the model’s performance, we propose incorporating the technique of Prompt Tuning or Prompt Learning in conjunction with MCM. While MCM is a zero-shot method, we propose utilizing prompt tuning to transform it into a few-shot approach, thereby enhancing its accuracy. The objective of prompt tuning is to optimize the prompts provided to the language model in order to minimize the loss. By employing our approach, we observed a notable enhancement of 1-2% in the results. Detailed coverage of both topics can be found in the subsequent chapters.

6.4 Organization of the Report

The introduction chapter of our report offers a comprehensive overview of the motivation behind selecting the topic, along with providing background information and outlining our specific contributions. Moving on, the second chapter delves into Out-Of-Distribution Detection, exploring its intricacies. Subsequently, the third chapter provides an extensive examination of Maximum Concept Matching (MCM). The fourth chapter is dedicated to discussing the Prompt Tuning/Learning approach that we have proposed. Finally, in the last chapter, we present the results obtained, draw conclusions based on our findings, and discuss potential future improvements to the model.

Chapter 7

Out-Of-Distribution Detection

7.1 Fundamentals of Out-Of-Distribution Detection

Out-of-distribution (OOD) detection plays a crucial role in the realm of machine learning systems operating in real-world environments. Its purpose is to identify samples that significantly deviate from the training data or lie beyond the known distribution. These samples can pose risks and challenges to the reliability and robustness of machine learning models, as they may result in erroneous predictions or unexpected behaviors.

In conventional machine learning scenarios, models are typically trained and evaluated on data that closely aligns with the distribution of the training set. However, in practical situations, models often encounter data that diverges from the training distribution. This can arise due to various factors, including the presence of novel or rare examples, deliberate adversarial attacks, changes in the data distribution (dataset shift), or data corruption. OOD samples can have a detrimental impact on model performance and may even lead to catastrophic failures if not accurately detected and handled.

To tackle these challenges, researchers have dedicated their efforts to the development of OOD detection methods. The main objective of OOD detection is to effectively differentiate between in-distribution samples and OOD samples. Traditional approaches to OOD detection frequently rely on statistical measures, such as uncertainty estimation, to identify samples exhibiting high model uncertainty or low confidence levels.

7.2 Applications of Out-Of-Distribution Detection

OOD detection offers extensive applications across diverse domains, contributing to the deployment of robust and reliable AI systems in the real world. One prominent domain where OOD detection is of utmost importance is autonomous driving. By accurately identifying objects or scenarios that the vehicle was not trained on, OOD detection enhances safety and helps prevent accidents.

In the field of healthcare, OOD detection plays a crucial role in detecting rare diseases or anomalies in medical images. By identifying patterns or features that deviate from the expected distribution, OOD detection aids in early diagnosis and effective treatment planning.

Another notable application of OOD detection lies in the realm of natural language processing. It facilitates the identification of out-of-domain queries, allowing systems to appropriately handle user inputs that fall outside their intended scope. Moreover, OOD detection can be employed in detecting fake news or misleading information, contributing to the promotion of reliable and accurate information dissemination.

In summary, the applications of OOD detection span across various domains, including autonomous driving, healthcare, and natural language processing. Its implementation empowers AI systems to operate with increased robustness and reliability, thereby enabling their effective utilization in real-world scenarios.

7.3 Approaches for Out-Of-Distribution Detection

Several popular Deep Learning techniques are commonly used for OOD Detection, such as Maximum Softmax Probability (MSP), Ensemble Learning, Temperature Scaling, ODIN, and SelectiveNet, among others.

7.3.1 Maximum Softmax Probability (MaxProb or MSP)

The maximum softmax probability (MaxProb) is a simple yet powerful technique used for out-of-distribution (OOD) detection in classification tasks. It operates by obtaining the highest softmax value among all classes, serving as an indicator of prediction confidence. In practice, correctly classified examples often exhibit higher MaxProb values compared to misclassified and OOD examples. This suggests that more confident predictions tend to be more accurate. MaxProb offers a straightforward approach to identify uncertain predictions, aiding in the detection of potential misclassifications and out-of-distribution samples. Its effectiveness and ease of implementation make it a valuable tool for improving the reliability and robustness of classification models, especially in scenarios where accurate confidence estimation is crucial for decision-making processes.

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}} \quad (7.1)$$

Correctly classified examples tend to have greater MaxProb than erroneously classified and out-of-distribution examples. In other words, more confident predictions indeed tend to be more accurate. [27]

7.3.2 Ensemble Learning

Ensemble learning is a powerful technique that involves the utilization of multiple models, all trained with the same learning algorithm, to collaboratively generate predictions for each data point. The primary objective of ensemble learning is to improve overall performance by combining the decisions made by these individual models. Various methods can be employed for combining the decisions:

Max Voting: The final prediction is determined by selecting the prediction that is most frequently agreed upon by the majority of the models.

Averaging: In regression tasks, the predictions of all models can be directly averaged, while in classification tasks, the softmax probabilities of the models can be averaged to obtain the final prediction confidence.

Weighted Averaging: each model is assigned individual weights, and a weighted average is computed based on these weights to derive the final prediction. These weights can be assigned according to the perceived reliability or expertise of each model.

Ensemble learning is widely used across diverse domains and has proven to be an effective strategy for boosting model accuracy, generalization, and robustness, making it a valuable tool for improving the performance of machine learning models in various applications.

7.3.3 Temperature Scaling

Temperature Scaling is an extension of Platt scaling designed to refine the prediction confidence obtained through the softmax function in techniques like MaxProb. In Temperature Scaling, a single scalar parameter 'T' greater than zero is introduced to compute the prediction confidence using a function 'q' (see Function 7.2)

By "softening" the softmax with $T \neq 1$, the output entropy is raised, leading to a slightly less confident network. This adjustment ensures that the confidence scores more accurately reflect true probabilities.

$$q(z_i) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}} \quad (7.2)$$

As T approaches infinity, the probabilities tend towards $1/J$, indicating maximum uncertainty. Conversely, when T is set to 1, the original softmax probabilities are recovered. The parameter T is learned through Negative Log-Likelihood on the validation set, optimizing the confidence calibration.

Notably, temperature scaling does not impact the model’s accuracy since it preserves the maximum of the softmax function, maintaining the class prediction probabilities while scaling all probabilities uniformly. This makes temperature scaling a valuable tool for refining prediction confidence and enhancing the reliability of classification models in various applications.

7.3.4 ODIN (Out-of-Distribution Detector for Neural Networks)

ODIN (Out-of-Distribution Detector for Neural Networks) is a state-of-the-art method for detecting out-of-distribution (OOD) samples in neural networks. OOD samples are data points that lie outside the training distribution and can lead to unreliable predictions. ODIN introduces a simple yet effective technique to enhance the model’s OOD detection capability. It leverages temperature scaling to "soften" the softmax output, making the network less confident in its predictions. Additionally, ODIN applies input perturbations during inference, enhancing the decision boundary for OOD samples. By computing the maximum softmax probability with these modifications, ODIN achieves significant improvements in OOD detection accuracy across various datasets and neural network architectures. This approach demonstrates the potential to enhance model robustness and reliability by providing more accurate uncertainty estimates and effectively identifying out-of-distribution samples, making it a valuable contribution to the field of deep learning.

ODIN reduces the false positive rate from the baseline 34.7% to 4.3% on the DenseNet (applied to CIFAR-10) when the true positive rate is 95% [28]

7.3.5 SelectiveNet

SelectiveNet is a novel and effective method designed for out-of-distribution (OOD) detection in deep learning models.

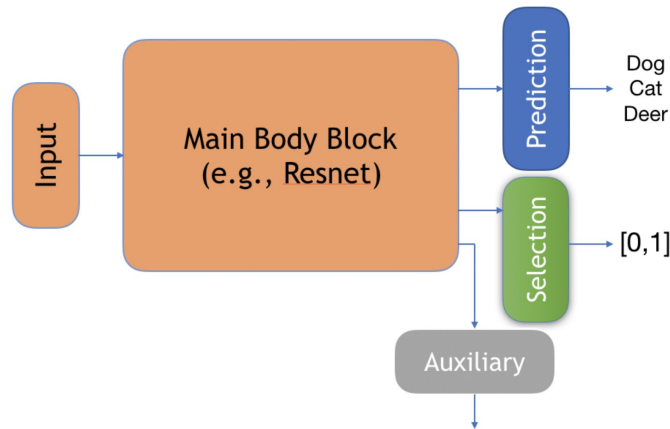


Figure 7.1: *SelectiveNet Basic Diagram* [6]

SelectiveNet introduces a selective classification approach, which dynamically chooses whether to accept or reject a prediction based on its confidence level. By incorporating an auxiliary network and an uncertainty-based gating mechanism (see Figure 7.1), SelectiveNet discriminatively selects in-distribution samples while effectively detecting and rejecting OOD samples. This selective classification strategy allows the model to boost its OOD detection performance significantly, outperforming several existing state-of-the-art methods. SelectiveNet provides an important contribution to enhancing the reliability and robustness of deep learning models, particularly in scenarios where accurate identification of OOD samples is critical for real-world applications.

7.4 Evaluation Metrics for Out-Of-Distribution Detection Methods

7.4.1 FAR95

FAR95 (False Alarm Rate at 95% True Negative Rate) is a fundamental evaluation metric used to assess the performance of out-of-distribution (OOD) detection methods.

The FAR95 is the probability that an in-distribution example raises a false alarm, assuming that 95% of all out-of-distribution examples are detected.

In the context of OOD detection, a false alarm occurs when an in-distribution sample is incorrectly classified as out-of-distribution. The FAR95 measures the proportion of such false alarms, specifically at a true negative rate of 95%. A higher FAR95 value indicates a higher rate of false alarms, which can be detrimental in applications where precision and reliability are crucial. Lower values of FAR95 are desirable as they reflect better OOD detection performance, meaning that the model is effectively distinguishing between in-distribution and out-of-distribution samples with a low rate of false alarms.

7.4.2 AUROC

AUROC (Area under Receiver Operating Characteristics) is a widely used evaluation metric for assessing the performance of out-of-distribution (OOD) detection methods. The Receiver Operating Characteristics (ROC) curve is a graphical representation of the true positive rate (sensitivity) against the false positive rate (1 - specificity) at various classification thresholds. (see Figure 7.2)

The AUROC provides a single scalar value that quantifies the overall capability of a model to discriminate between in-distribution and out-of-distribution samples. A higher AUROC value indicates better OOD detection performance, with the model achieving a higher true positive rate while maintaining a lower false positive rate.

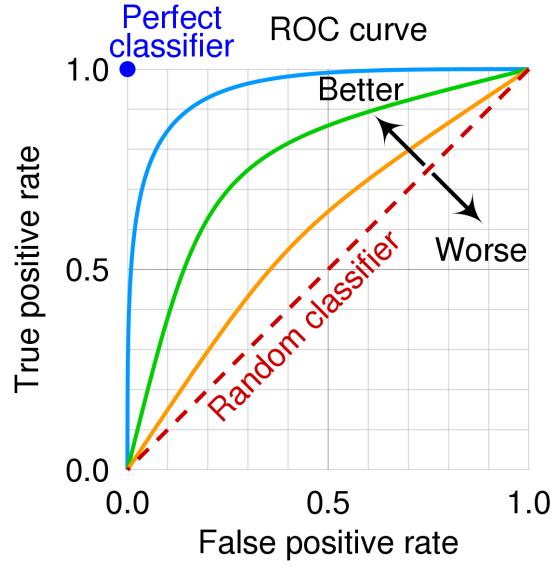


Figure 7.2: *ROC Curve*

AUROC is especially useful for comparing different OOD detection methods and selecting the most effective ones for specific tasks. As a comprehensive metric, AUROC complements other evaluation measures and helps gain valuable insights into the overall quality of OOD detection models.

$$\text{True Positive Rate} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}} \quad (7.3)$$

$$\text{False Positive Rate} = (1 - \text{specificity}) = \frac{\text{FP}}{\text{TN} + \text{FP}} \quad (7.4)$$

7.4.3 AUPR

AUPR (Area under Precision-Recall Curve) is a vital evaluation metric used to assess the performance of out-of-distribution (OOD) detection methods. The Precision-Recall curve plots the precision (positive predictive value) against the recall (true positive rate) at various classification thresholds. (see Figure 7.3)

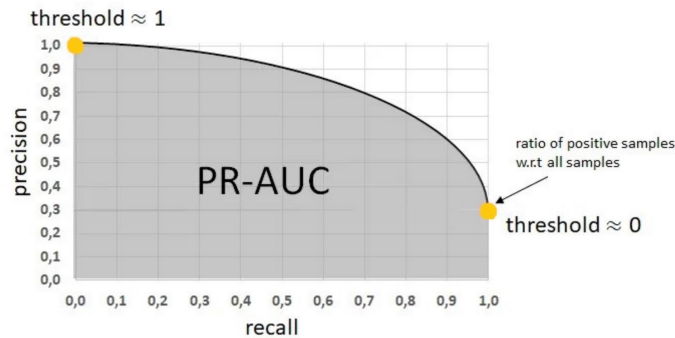


Figure 7.3: *Precision-Recall Curve*

The AUPR represents the integral of this curve and provides a single scalar value that measures the overall effectiveness of a model in detecting OOD samples. A higher AUPR value indicates better OOD detection performance, with the model achieving higher precision while maintaining a higher recall rate. AUPR is particularly valuable when dealing with imbalanced datasets, which is common in OOD detection scenarios. It allows us to gain insights into the trade-off between precision and recall and select the most suitable OOD detection methods for specific applications. By complementing other evaluation metrics like AUROC, AUPR provides a comprehensive view of the model’s performance.

ROC curves are appropriate when the observations are balanced between each class, whereas precision-recall curves are appropriate for imbalanced datasets.

$$\text{Recall} = \text{True Positive Rate} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (7.5)$$

$$\text{Precision} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}} \quad (7.6)$$

7.4.4 FPR95

In the realm of Out-of-Distribution (OOD) detection, the FPR95 (False Positive Rate at 95% True Positive Rate) metric plays a pivotal role in assessing the reliability and effectiveness of OOD detection models. As OOD samples pose a significant challenge due to their departure from the training data distribution, it becomes crucial to identify them accurately while maintaining a satisfactory level of true positive rate (TPR) for in-distribution data. *FPR95 measures the percentage of OOD samples incorrectly classified as in-distribution data when the model attains a true positive rate of 95% for in-distribution samples.*

Chapter 8

Maximum Concept Matching (MCM)

8.1 Description of MCM

Maximum Concept Matching (MCM) is a method proposed for out-of-distribution (OOD) detection, which plays a critical role in machine learning systems deployed in the open world. Traditional OOD detection methods have been largely unimodal, focusing solely on vision or language modalities, thereby overlooking the valuable information available in multi-modal representations. Inspired by the remarkable achievements of vision-language pre-training, MCM seeks to revolutionize OOD detection by transitioning from a single-modal to a multi-modal paradigm.

At its essence, MCM is a straightforward yet highly effective zero-shot OOD detection approach that aligns visual features with textual concepts. Leveraging the knowledge acquired through pre-training on vision-language tasks using models such as CLIP, MCM enables the model to grasp the intricate relationships between visual and textual data. By aligning the visual features of an input sample with the corresponding textual concepts, MCM can accurately determine if the sample belongs to the expected distribution or if it is an OOD sample. This innovative fusion of multi-modal information equips MCM with the ability to significantly enhance the performance and reliability of OOD detection, thereby contributing to the advancements in deploying machine learning models in real-world scenarios.

8.2 Advantages of MCM

The advantage of MCM lies in its effective utilization of multi-modal representations, harnessing the complementary information offered by both visual and textual data. By aligning visual features with textual concepts, MCM exploits the underlying semantic relationships and synergies between vision and language. This results in a more robust

and comprehensive understanding of the data, leading to improved OOD detection performance.

The advantages of Maximum Concept Matching (MCM) [7] because of the strong pre-trained CLIP model are:

Generalizable to many tasks: Unlike traditional OOD detection methods that rely on task-specific models, MCM demonstrates the ability to perform various OOD detection tasks with just one single model. For new tasks, only the names of the task’s visual concepts are required, making it highly versatile and adaptable.

OOD-agnostic: MCM is OOD-agnostic, as it does not rely on any OOD information, making it well-suited for real-world scenarios where unknowns cannot be anticipated in advance. This addresses the shortcomings of certain approaches that assume knowledge of unseen labels as weak information about OOD data.

Training-free: MCM enables OOD detection in a zero-shot fashion. MCM enables OOD detection in a training-free manner, as it uses large pre-trained models like CLIP, setting it apart from a majority of OOD detection literature that requires training from scratch or fine-tuning to achieve competitive performance.

Scalable: The contrastive vision-language pre-training makes MCM scalable to a large number of class labels and realistic high-resolution images, making it an attractive and powerful solution for various OOD detection applications.

On ImageNet-1k, MCM achieves an average AUROC of 91.49%, outperforming methods that require training. [7]

8.3 MCM Approach

The Maximum Concept Matching approach utilizes the CLIP model to calculate the MCM Score. The Image and Text encoders in the CLIP model are employed to extract image and text features, respectively. The images are passed through the Image encoder to obtain visual features, while a text prompt constructed using the class label is fed into the Text encoder to acquire language features. Subsequently, the cosine similarity between the image feature and the language features (for each class) is computed. To obtain the final MCM score, a Softmax function with temperature scaling is applied to the obtained cosine similarities. This temperature scaling ”softens” the output, resulting in a more calibrated confidence estimation. The MCM score is then determined by

selecting the maximum softmax value among all classes, providing a measure of the prediction confidence for the given input sample. This process of utilizing the CLIP model and applying similarity calculations along with temperature scaling empowers the MCM approach with the ability to effectively align visual and textual information, leading to improved out-of-distribution (OOD) detection performance.

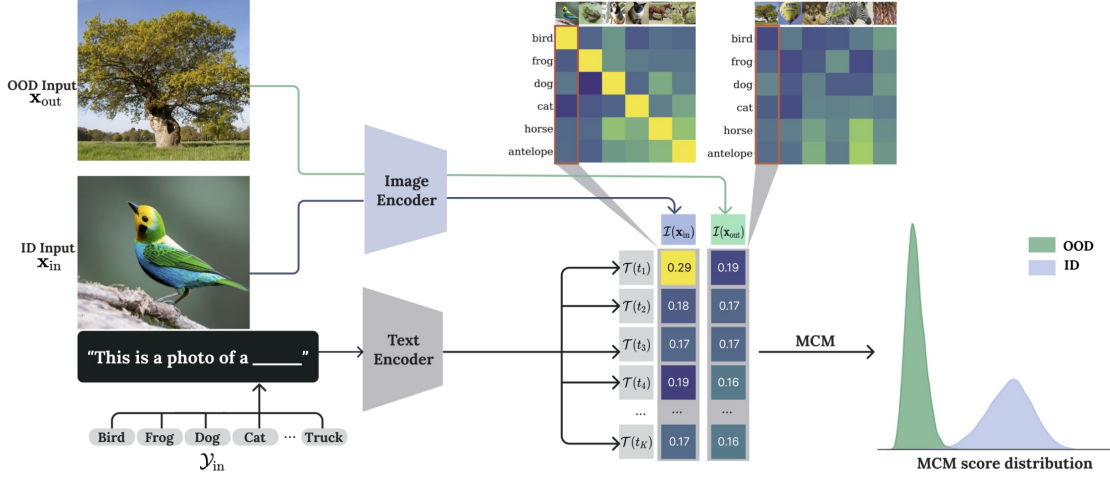


Figure 8.1: Zero-shot OOD detection framework using MCM and CLIP model. [7]

Lets see how the MCM Score is calculated mathematically (Refer to Figure 8.1):

The ID classification task is defined by a set of class labels $\mathcal{Y}_{in}$. The goal of OOD detection is to detect samples that do not belong to $\mathcal{Y}_{in}$. We view the textual embeddings of ID classes (wrapped by text templates) as concept prototypes. The OOD uncertainty of an input image can be characterized by the distance from visual features to the closest ID prototype. By properly scaling the distance, the MCM score achieves strong ID-OOD separability. [7]

Let $\mathcal{Y}_{in} = \{y_1, y_2, \dots, y_K\}$ be the set of class labels, then we can create a set of concept vectors $\mathcal{T}(t_i), i \in \{1, 2, \dots, K\}$, where t_i is the text prompt "this is a photo of a $\langle y_i \rangle$ " for a label y_i . The concept vectors are the embeddings of the text prompts t_i i.e. the output of text encoder of the CLIP model for the text prompt t_i . Let $\mathcal{I}(\mathbf{x}')$ be the Image feature of the input image $\mathbf{x}'$. Then we can compute the label-wise matching score by evaluating the cosine similarity between the image feature $\mathcal{I}(\mathbf{x}')$ and the concept vector $\mathcal{T}(t_i)$:

$$s_i(\mathbf{x}') = \frac{\mathcal{I}(\mathbf{x}') \cdot \mathcal{T}(t_i)}{\|\mathcal{I}(\mathbf{x}')\| \cdot \|\mathcal{T}(t_i)\|}$$

Where $s_i(\mathbf{x}')$ is the Cosine Similarity of the features of class y_i and image $\mathbf{x}'$.

The maximum concept matching (MCM) score is defined as:

$$S_{\text{MCM}}(\mathbf{x}'; \mathcal{Y}_{\text{in}}, \mathcal{T}, \mathcal{I}) = \max_i \frac{e^{s_i(\mathbf{x}')/\tau}}{\sum_{j=1}^K e^{s_j(\mathbf{x}')/\tau}} \quad (8.1)$$

where τ is the temperature for softmax scaling.

Any image from in-distribution (ID) data (image embedding) will be matched to one of the concept vectors (text embedding). As a result, we obtain a high MCM score, indicating a strong confidence in the prediction. Conversely, for out-of-distribution (OOD) data, the MCM score will be low, signifying lower confidence in the prediction. In summary, the MCM score serves as a reliable indicator, with high scores corresponding to in-distribution data and low scores corresponding to out-of-distribution data.

Hence, the OOD detection function is formulated as:

$$G(\mathbf{x}'; \mathcal{Y}_{\text{in}}, \mathcal{T}, \mathcal{I}) = \begin{cases} 1 & S_{\text{MCM}}(\mathbf{x}'; \mathcal{Y}_{\text{in}}, \mathcal{T}, \mathcal{I}) \geq \lambda \\ 0 & S_{\text{MCM}}(\mathbf{x}'; \mathcal{Y}_{\text{in}}, \mathcal{T}, \mathcal{I}) < \lambda \end{cases} \quad (8.2)$$

where 1 represents in-distribution (ID) and 0 indicates out-of-distribution (OOD). λ is chosen such that a high fraction of ID data (e.g. 95%) is above the threshold.

8.4 Results: MCM

8.4.1 Our results

In our study, we evaluated the OOD detection performance using the CLIP model and MCM method. The evaluation encompassed tests conducted on two in-distribution (ID) datasets and four out-of-distribution (OOD) datasets.

- ID datasets:
 - The Oxford-IIIT Pet Dataset (pet37) [29]
 - Caltech-UCSD Birds-200-2011 Dataset (bird200) [30]
- OOD Datasets:
 - Scene UNDERstanding (SUN)
 - Describable Textures Dataset (DTD)
 - iNaturalist Dataset (iNat)
 - The Places365-Standard dataset (places365)

Results for pet37 as ID dataset

We calculated the MCM scores using CLIP model for pet37 as ID dataset. The results are visualized in Figure 8.2, with the x-axis representing the MCM score and the y-axis indicating the density of the samples (images). The plot illustrates distinct distributions, where the green distribution corresponds to out-of-distribution (OOD) samples and the blue distribution represents in-distribution (ID) samples. As observed, OOD samples tend to exhibit lower MCM scores, while ID samples have higher MCM scores, leading to minimal overlap between the two distributions. A reduced overlap between ID and OOD distributions indicates a more accurate model and predictions, highlighting the effectiveness of the MCM method in distinguishing between in-distribution and out-of-distribution samples.

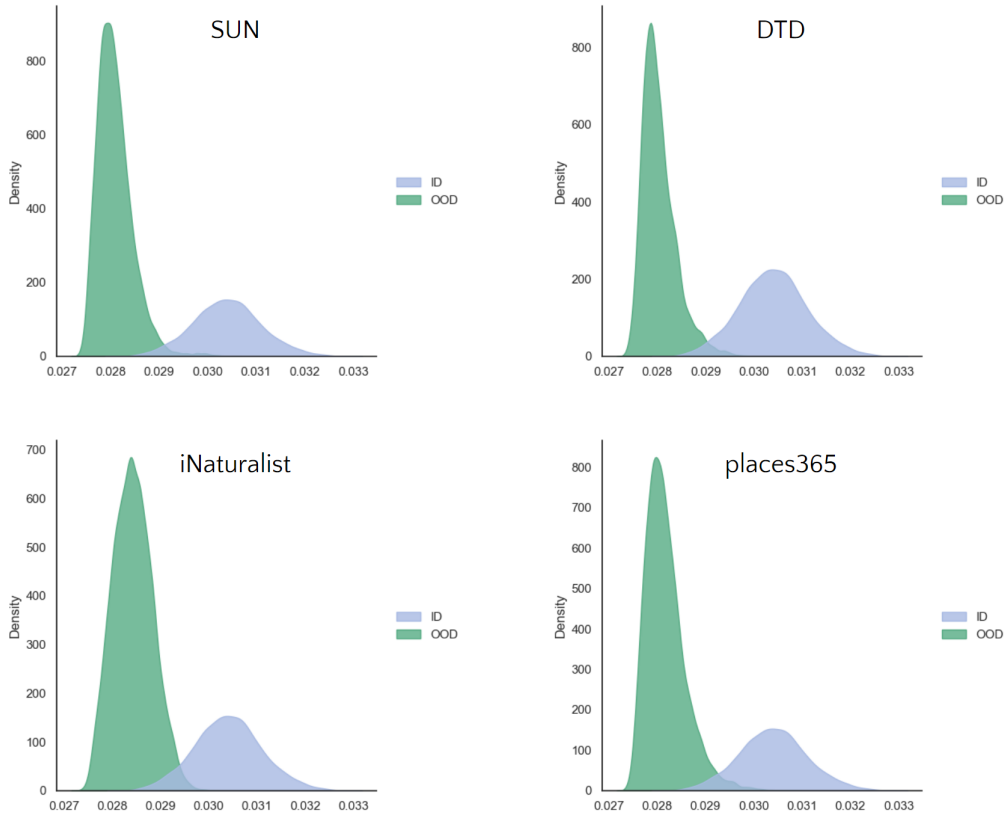


Figure 8.2: *MCM scores for Pet37 as ID Dataset with various OOD Datasets*

The results of our experiments, as depicted in Figure 8.3, align closely with the scores reported by the authors of the MCM paper. The figure illustrates the FPR95, AUPR, and AUROC scores, and our findings are in agreement with the outcomes presented in the original MCM research [7]. This consistency in results further validates the robustness and reliability of our experimental evaluation, affirming the effectiveness of the MCM approach for OOD detection in our study.

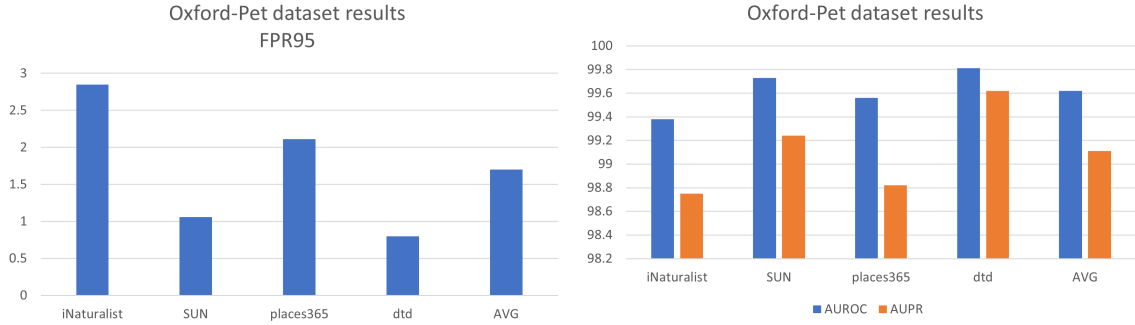


Figure 8.3: *FPR95, AUROC and AUPR scores for Pet37 as ID Dataset with various OOD Datasets*

Results for bird200 as ID dataset

In addition, we conducted further testing of the MCM method using bird200 as the in-distribution (ID) dataset, and the corresponding MCM scores can be observed in Figure 8.4. Once again, the two distributions are distinctly separated, affirming the efficacy of the MCM approach. The results of this evaluation are presented in Figure 8.5, where the FPR95, AUPR, and AUROC scores are displayed.

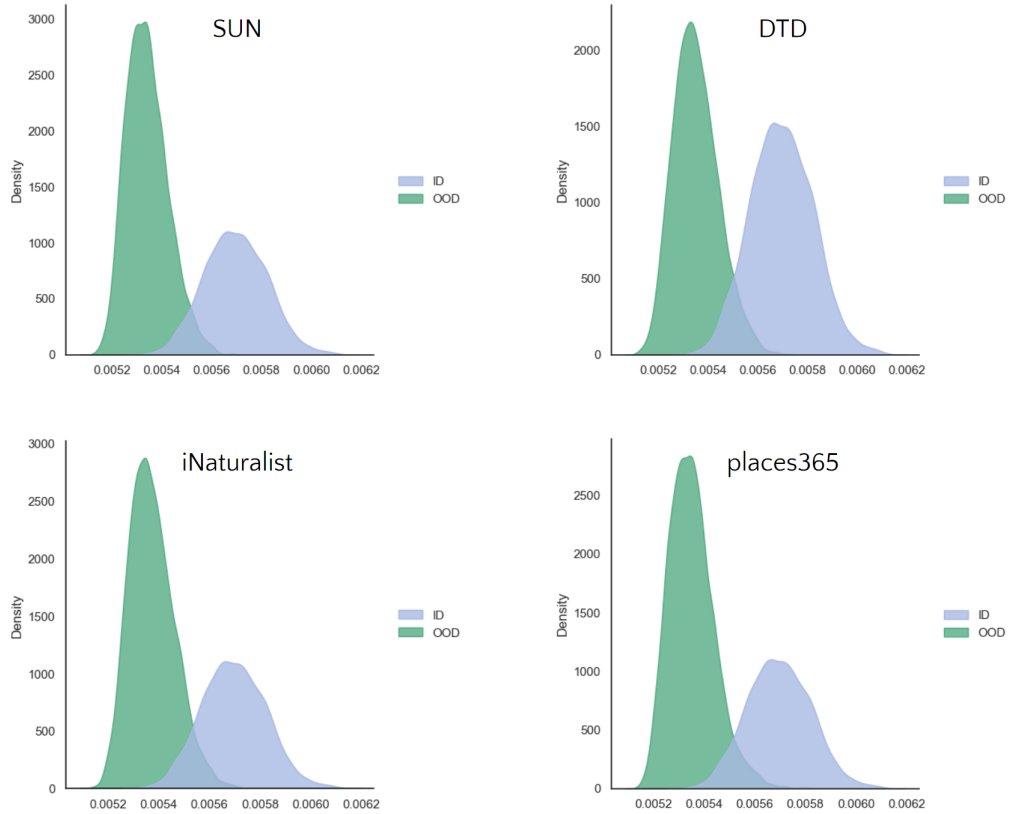


Figure 8.4: *MCM scores for bird200 as ID Dataset with various OOD Datasets*

Notably, we observed a slightly reduced performance in the case of iNaturalist as the out-of-distribution (OOD) dataset and bird200 as the ID dataset. Specifically, the FPR95 score is higher (indicating worse performance) compared to other OOD datasets, and the

AUROC/AUPR scores are lower (indicating worse performance) compared to the other three OOD datasets. This discrepancy in performance can be attributed to the similarity in images between bird200 and iNaturalist datasets, both containing images with green backgrounds, trees, and natural landscapes. Due to this resemblance, the MCM scores exhibit a slight overlap for iNaturalist, leading to slightly diminished OOD detection performance compared to the other OOD datasets.

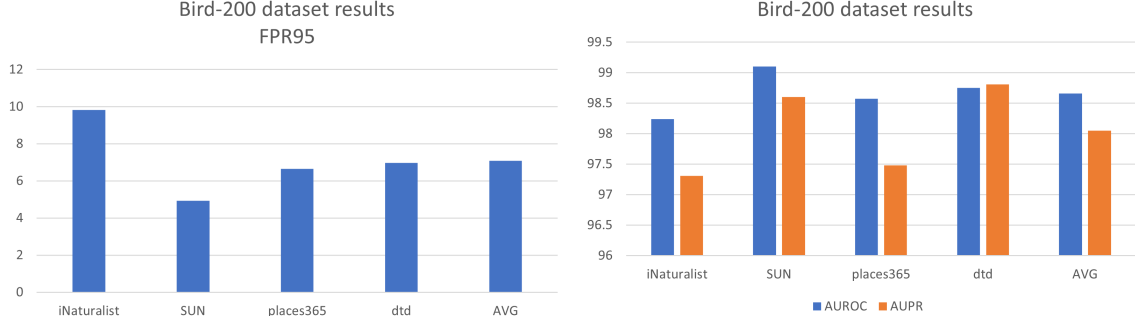


Figure 8.5: *FPR95, AUROC and AUPR scores for bird200 as ID Dataset with various OOD Datasets*

Results for various dataset

Table 8.1: Results of MCM OOD detection using CLIP on different datasets.

ID Dataset	OOD Dataset									
	iNaturalist		SUN		Places		Texture		Average	
	FPR95 ↓	AUROC ↑	FPR95 ↓	AUROC ↑	FPR95 ↓	AUROC ↑	FPR95 ↓	AUROC ↑	FPR95 ↓	AUROC ↑
CUB-200 [30]	9.83	98.24	4.93	99.10	6.65	98.57	6.97	98.75	7.09	98.66
Stanford-Cars [31]	0.05	99.77	0.02	99.95	0.24	99.89	0.02	99.96	0.08	99.89
Food-101 [32]	0.64	99.78	0.90	99.75	1.86	99.58	4.04	98.62	1.86	99.43
Oxford-Pet [29]	2.85	99.38	1.06	99.73	2.11	99.56	0.80	99.81	1.70	99.62
ImageNet-10	0.12	99.80	0.29	99.79	0.88	99.62	0.04	99.90	0.33	99.78
ImageNet-20	1.02	99.66	2.55	99.50	4.40	99.11	2.43	99.03	2.60	99.32
ImageNet-100	18.13	96.77	36.45	94.54	34.52	94.36	41.22	92.25	32.58	94.48

In Table 8.1, we present the FPR95 and AUROC scores obtained for different in-distribution (ID) and out-of-distribution (OOD) datasets. To ensure consistency, we used the same Random seed as the authors of the MCM [7] research paper when conducting our experiments, and our results closely align with theirs. As observed, the performance gradually decreases when using large ID datasets such as Imagenet-10, 20, and 100, which is an expected outcome. This decrease in performance is also observed in other OOD detection methods. The impact of using large ID datasets on OOD detection performance is a common observation and underscores the challenges posed by scaling to larger datasets in this context.

8.4.2 Comparison

Table 8.2 presents a comprehensive comparison of various OOD detection methods on ImageNet-1k [33] as ID Dataset. Notably, the MCM method stands out as it surpasses

many other OOD methods that require model training or fine-tuning. Despite being a zero-shot approach, meaning it does not require training, the average performance of MCM is superior to almost all other OOD methods, except the Energy method.

Furthermore, the performance of MCM can be further enhanced by utilizing larger models. For instance, when based on CLIP-L (the larger model), MCM achieves a remarkable 4.57% reduction in the False Positive Rate at 95% True Negative Rate (FPR95) compared to CLIP-B (the smaller model). Additionally, the zero-shot in-distribution (ID) classification accuracy is improved by 6.27%, reaching an impressive 73.28%. This observation indicates that larger models exhibit superior representation quality, benefiting both ID classification and OOD detection using the MCM method. The findings underscore the advantages of employing larger models in OOD detection, leading to enhanced performance and more accurate classification results.

Table 8.2: Comparison of MCM with other OOD detection methods. [7]

Method	iNaturalist		SUN		OOD Dataset Places		Texture		Average	
	FPR95 ↓	AUROC ↑	FPR95 ↓	AUROC ↑	FPR95 ↓	AUROC ↑	FPR95 ↓	AUROC ↑	FPR95 ↓	AUROC ↑
<i>Requires training (or w. fine-tuning)</i>										
MOS [34] (BiT)	9.28	98.15	40.63	92.01	49.54	89.06	60.43	81.23	39.97	90.11
Fort et al. [35] (ViT-B)	15.07	96.64	54.12	86.37	57.99	85.24	53.32	84.77	45.12	88.25
Fort et al. [35] (ViT-L)	15.74	96.51	52.34	87.32	55.14	86.48	51.38	85.54	43.65	88.96
Energy [36] (CLIP-B)	21.59	95.99	34.28	93.15	36.64	91.82	51.18	88.09	35.92	92.26
Energy [36] (CLIP-L)	10.62	97.52	30.46	93.83	32.25	93.01	44.35	89.64	29.42	93.50
MSP [37] (CLIP-B)	40.89	88.63	65.81	81.24	67.90	80.14	64.96	78.16	59.89	82.04
MSP [37] (CLIP-L)	34.54	92.62	61.18	83.68	59.86	84.10	59.27	82.31	53.71	85.68
<i>Zero-shot (no training required)</i>										
MCM [7] (CLIP-B)	30.91	94.61	37.59	92.57	44.69	89.77	57.77	86.11	42.74	90.77
MCM [7] (CLIP-L)	28.38	94.95	29.00	94.14	35.42	92.00	59.88	84.88	38.17	91.49

8.4.3 Conclusion

In this study, we venture into uncharted territory for OOD detection, departing from the conventional single-modal approach and embracing a multi-modal perspective. Our investigation centers around a OOD detection method called MCM, which leverages joint vision-language representations and treats textual aspects as "concept prototypes." Distinct from the majority of OOD detection techniques, MCM offers several compelling advantages: it does not necessitate any prior training, demonstrates generalizability across diverse tasks, scales effectively to handle numerous classes, and provides comprehensive details about OOD inputs. These distinctive features make MCM an attractive and promising approach for OOD detection, opening new possibilities in the domain of multi-modal representation learning and OOD detection strategies.

Chapter 9

Prompt Learning/Tuning

9.1 Introduction to Prompt Learning

Prompt learning is a dynamic training and fine-tuning process designed for language or vision-language models. It revolves around learning the prompts, which are represented by a set of learnable vectors, that serve as inputs to the language model, ultimately influencing the model’s output. The primary goal of prompt learning is to optimize and refine these prompts to improve the model’s performance on specific tasks or datasets.

A key advantage of prompt learning lies in its resource-efficient nature. Unlike traditional fine-tuning approaches, prompt learning avoids making modifications to the pretrained model’s weights, which can be computationally expensive. Instead, it focuses solely on tuning the prompts, requiring less labeled data compared to conventional fine-tuning methods. This approach allows us to tailor the model’s behavior and adapt it to specific tasks without the need for extensive changes to the underlying model architecture.

In our MCM method, we initially used "Hard prompts," which involved manually engineering prompts in the form of 'a photo of a $< y_i >$ '. However, we recognize the potential of prompt learning/tuning to further enhance our model’s performance. By applying prompt learning with the CLIP model, we can generate "Soft prompts," which are learned prompts rather than hand-crafted ones. This opens up new avenues for improving the effectiveness of our MCM approach as the model learns to optimize and adapt the prompts based on the data and task at hand. The utilization of soft prompts is expected to unlock additional performance gains, taking our OOD detection model to greater heights of accuracy and adaptability.

"Prompt-tuning is an efficient, low-cost way of adapting an AI foundation model to new downstream tasks without retraining the model and updating its weights."

—IBM

9.2 Methods for Prompt Learning

9.2.1 Context Optimization (CoOp)

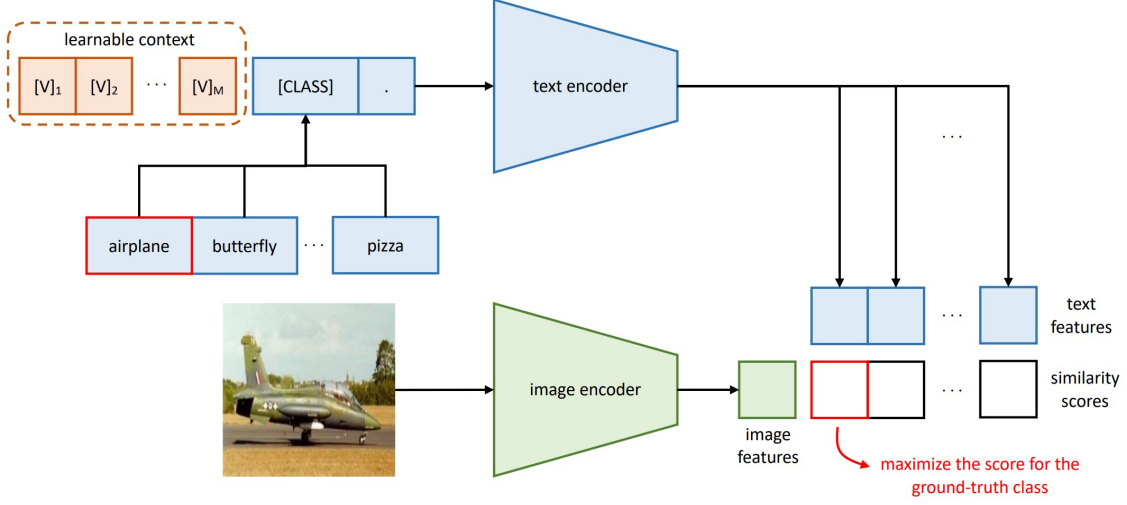


Figure 9.1: *CoOp Architecture* [8]

The deployment of large pre-trained vision-language models such as CLIP has shown promising results in knowledge transfer across various tasks. However, a major bottleneck in utilizing these models lies in prompt engineering, a time-consuming process involving fine-tuning to optimize performance. To address this challenge, we leverage the Context Optimization (CoOp) method, a straightforward yet effective approach for adapting CLIP-like models specifically for image recognition tasks. CoOp achieves this by modeling a prompt’s context words with learnable vectors while keeping the pre-trained parameters fixed, making it a resource-efficient solution.

Through extensive experiments conducted on 11 datasets [8], CoOp has demonstrated its superiority over hand-crafted prompts with minimal shots, and it showcases significant performance improvements with more shots. Despite being a learning-based approach, CoOp exhibits remarkable domain generalization performance, outperforming zero-shot models that rely on hand-crafted prompts. This highlights the robustness and versatility of CoOp in adapting to various domains and tasks without extensive fine-tuning, making it a valuable addition to the arsenal of prompt engineering techniques for vision-language models like CLIP.

The CoOp architecture, as illustrated in Figure 9.1, is employed for image recognition tasks, where it effectively learns prompt vectors by maximizing the cosine similarity with the ground truth (true class label). To further enhance the CoOp method for OOD detection, we have made slight modifications to incorporate the MCM approach. This integration allows the CoOp framework to benefit from the robustness and effectiveness of

MCM in distinguishing between in-distribution and out-of-distribution samples, creating a comprehensive and powerful solution for both image recognition and OOD detection tasks.

9.2.2 Conditional Context Optimization (CoCoOp)

CoOp, while effective, faces a critical challenge of overfitting to observed base classes, resulting in limited generalizability to unseen classes within the same dataset. To address this limitation and further enhance the model’s performance, Context Optimization (CoCoOp) is used. CoCoOp takes inspiration from CoOp but introduces a novel approach to dynamically generate input-conditional tokens for each image. This dynamic prompt generation allows for better generalization to unseen classes and significantly improves domain generalization performance.

CoCoOp achieves this by incorporating a lightweight neural network known as Meta-Net, which generates input-conditional tokens from the image encoding produced by the CLIP model. These tokens are then combined with the prompt vectors from the CoOp method to form the final soft prompts. The integration of Meta-Net enables the creation of dynamic prompts that are less sensitive to class shifts, resulting in improved robustness and adaptability of the CoCoOp model.

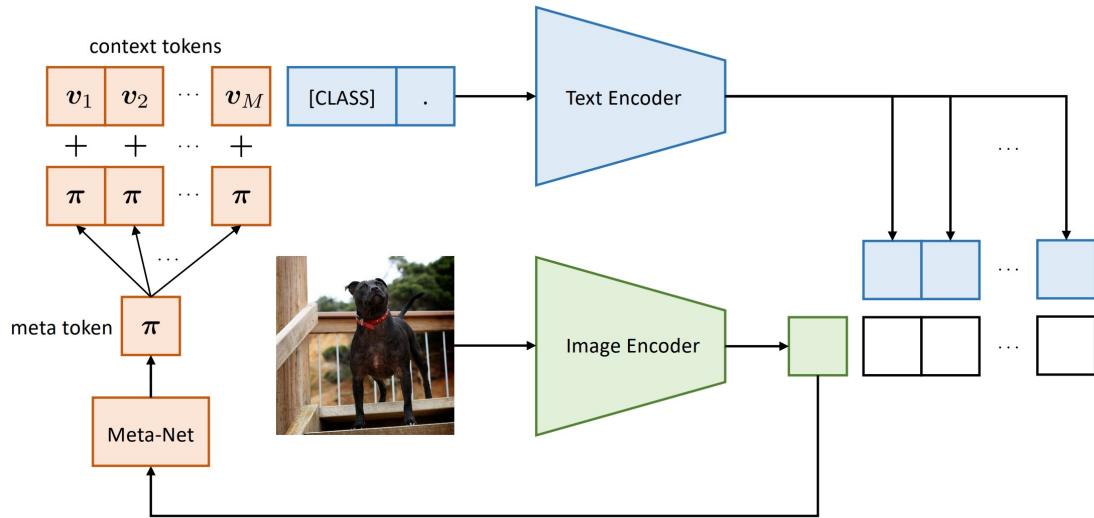


Figure 9.2: *CoCoOp Architecture* [9]

As depicted in Figure 9.2, CoCoOp builds upon the foundation of the CoOp method, leveraging the generated tokens from Meta-Net to augment the prompt vectors. This innovative approach significantly enhances the model’s ability to generalize to unseen classes within the dataset, providing a more comprehensive and powerful solution for image recognition tasks.

Although our testing has focused solely on the CoOp method, the CoCoOp+MCM method is expected to deliver even more impressive results than the CoOp+MCM approach we have employed. Future studies and evaluations should be conducted to validate the potential of CoCoOp+MCM, which holds promise in further enhancing the OOD detection performance and overall effectiveness of our approach. With the integration of CoCoOp’s dynamic prompts and the robustness of MCM, we anticipate a more powerful and versatile solution for image recognition and OOD detection tasks.

Chapter 10

Results: OOD Detection

10.1 Hyperparameter Selection

In our study, which follows a few-shot approach, we explored varying numbers of training examples per class, ranging from 2 to 16. We opted to go with 4 examples per class, as it was found to yield optimal results in the referenced research paper [8].

Furthermore, we have the flexibility to determine the position of the class label vector/token within the prompt. We tested both middle and end positions and observed the end position to be more favorable for our specific task.

The number of prompt vectors or context can also be adjusted to influence the model's behavior. Through experimentation, we examined the impact of different prompt vector counts and observed that having 16 context vectors led to superior performance.

Moreover, instead of using the same prompt for all classes, we explored the method called Class-Specific Context (CSC), which involves learning distinct prompts for each class. We evaluated the effects of CSC and its variations on the model's performance.

The Figure 10.1 illustrates the results of these parameter changes. From the graph, we observed that the model represented by the green line exhibited the best performance. Based on these findings, we made our hyperparameter selections, settling on 16 context vectors with an end position for the class label vector and without employing CSC.

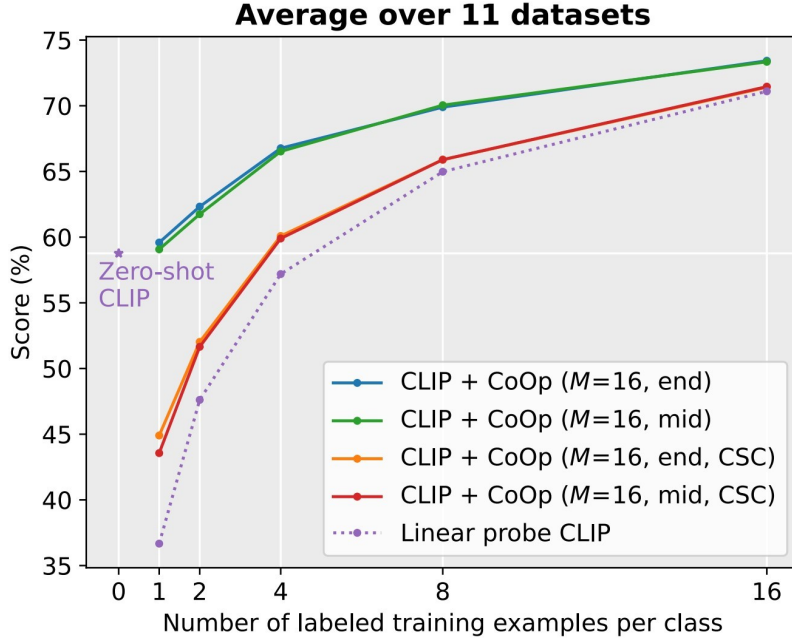


Figure 10.1: *Result of CLIP+CoOp method tested on 11 datasets.* [8]

M = number of context vectors (prompt words)

Mid, end = position of the class vector in the prompt.

CSC = Learn different context for different classes rather than using the same context.

10.2 Training

We integrated the CoOp method into the code of the MCM model. The training process involved running the model for 200 epochs. As we adopted a 4-shot approach, utilizing 4 training examples for each class, the total iterations for 200 epochs amounted to 800. For our experiment, we utilized the Oxford pet37 [29] as the in-distribution (ID) dataset.

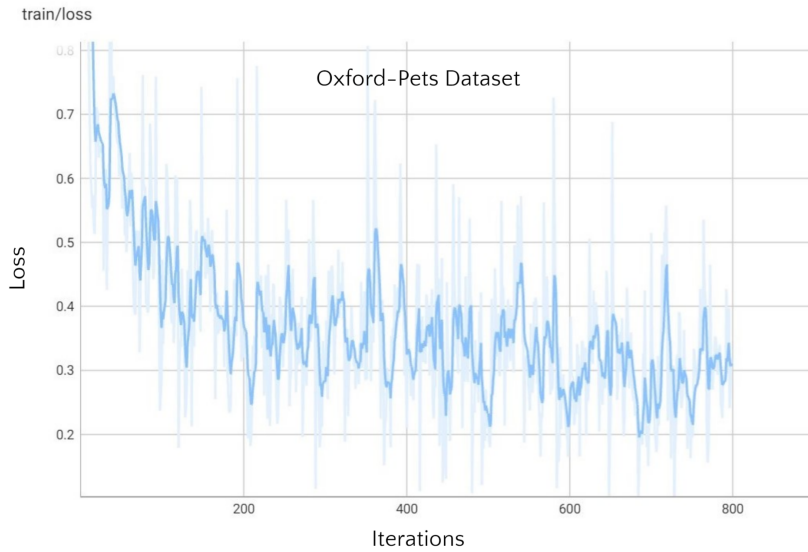


Figure 10.2: *Training Loss for CLIP+MCM+CoOp model*

The loss curve of our model during training is illustrated in Figure 10.2. Upon observing the graph, it becomes evident that the loss is decreasing exponentially. Although we trained the model for 200 epochs due to limitations in time and resources, the trend indicates that the loss is expected to continue decreasing further with additional epochs. The smoothing out of the graph highlights the gradual improvement in the model’s performance over the training process, suggesting that training for more epochs could potentially yield even better results.

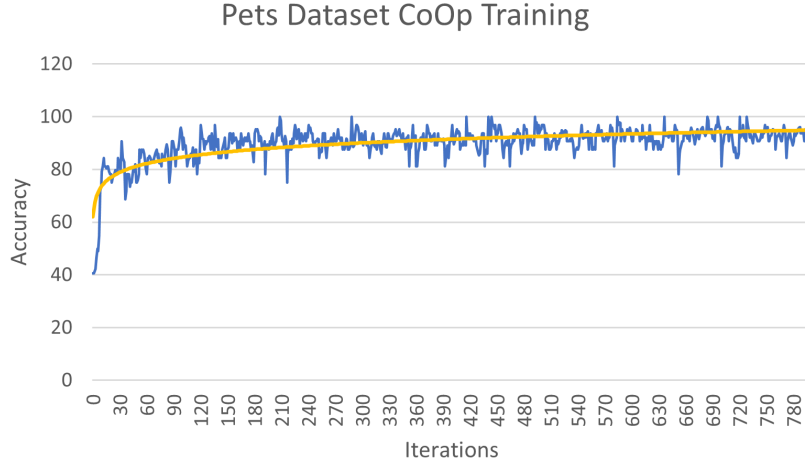


Figure 10.3: *Training Accuracy for CLIP+MCM+CoOp model*

Figure 10.3 depicts the classification accuracy of our model over the course of training. Notably, the accuracy demonstrates a clear upward trend as the number of epochs/iterations increases. Upon training the model for 200 epochs, we achieved a training accuracy of 92.96%. Although further improvements are expected to be marginal, it is reasonable to anticipate a slight increase in accuracy with additional epochs.

10.3 Results

Evaluation (image classification) of CLIP+MCM+CoOp model after 200 epochs:

- **Total Samples:** 3,669
- **Correct Samples:** 3,386
- **Accuracy:** 92.3%
- **Error:** 7.7%

The soft prompts learned through the CoOp method are represented as vectors. To discern the words that best represent these tokens/prompts, we can compute the words that are in close proximity to these vectors. As part of our analysis, we visualized the three

nearest words for each of the 15 learned context tokens. The results of this visualization are presented below in Figure 10.4:

1.	lindsey	sikhs	generates
2.	artist	losers	affirmation
3.	mates	samuel	caretaker
4.	sh	owing	mags
5.	leftovers	margarita	victoria
6.	semis	relent	mo
7.	pedi	tract	gull
8.	turns	decrease	H
9.	ōŁĴĥ	incre	ewi
10.	daytona	totem	happyhalloween
11.	flaw	van	els
12.	shane	aam	elle
13.	anic	marketplace	ministries
14.	mbles	trainees	startrek
15.	Ö	cracks	yÃ¼
16.	owl	<-Class Label	

Figure 10.4: *Visualizing 3 nearest words for the 15 learned context tokens*

The learned soft prompts are employed to compute the MCM scores for OOD detection. Figure 10.5 presents the results of our experiment, utilizing pet37 [29] as the ID dataset and DTD as the OOD dataset. Upon closer examination, we observe that the OOD and ID distributions exhibit greater separation in the 'soft prompt' graph compared to the 'hard prompt' graph. This significant distinction suggests that our method, MCM+Prompt Learning, performs slightly better than MCM alone. The incorporation of soft prompts enhances the model's ability to distinguish between in-distribution and out-of-distribution samples, leading to improved OOD detection performance.

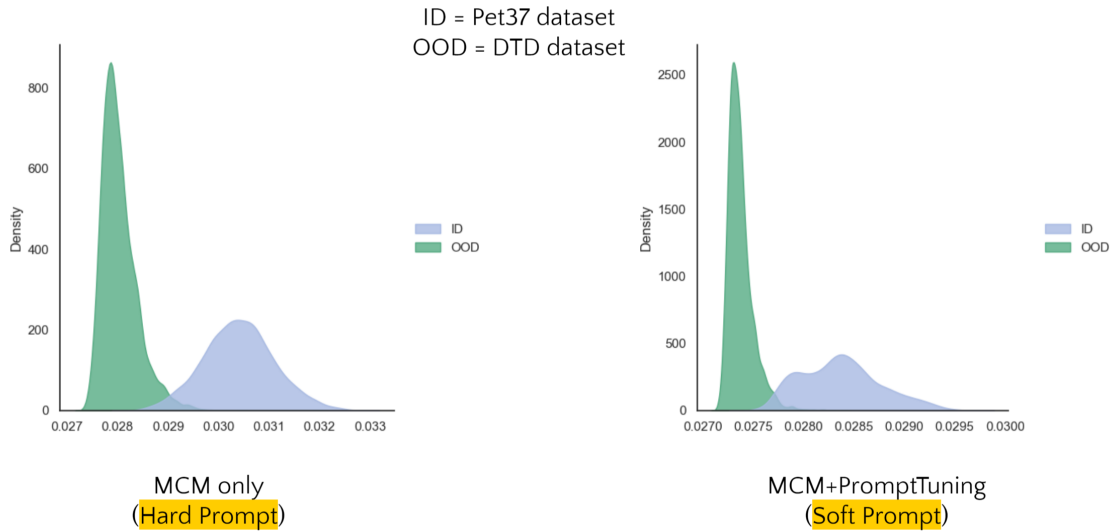


Figure 10.5: *Visualizing 3 nearest words for the 15 learned context tokens*

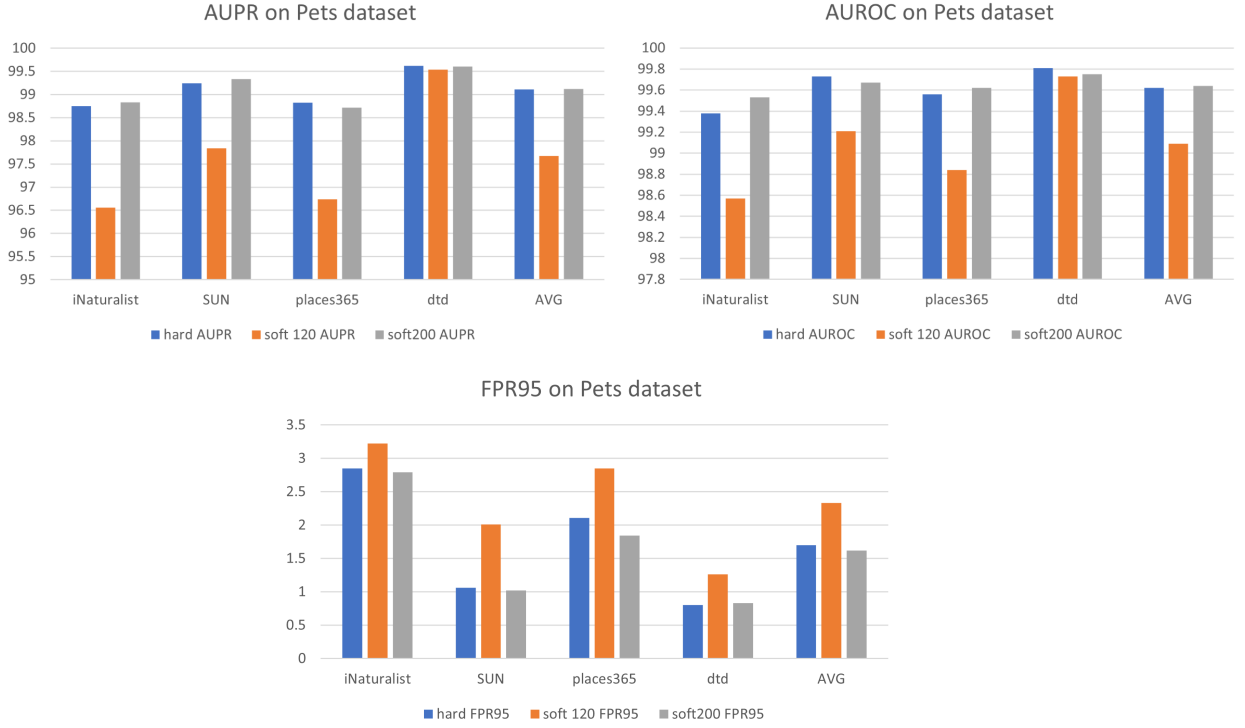


Figure 10.6: *OOD Detection Results of our model (MCM + Prompt Learning) for various OOD datasets*

In Figure 10.6, we present the AUPR, AUROC, and FPR95 scores obtained from our experiments on our model (MCM + Prompt Learning). The ID dataset used for the experiments is pet37 [29], while the OOD datasets comprise iNaturalist, SUN, Places365, and DTD. The results indicate that our proposed approach utilizing soft prompts outperforms the conventional MCM approach with hard prompts. Notably, if the model is trained for 120 epochs, its performance is inferior to the MCM method; however, with 200 epochs of training, our approach exhibits better performance than the MCM method. These findings highlight the significance of prompt learning using soft prompts in improving OOD detection capabilities and demonstrate the impact of the training duration on the model’s effectiveness.

10.4 Conclusion

In this study, we ventured into the realm of multi-modal Deep Learning for out-of-distribution (OOD) detection, diverging from conventional single-modal methods. Our focus was on exploring the potential of MCM, an OOD detection approach that harnesses joint vision-language representations. MCM exhibited numerous advantages, including its training-free nature, applicability to diverse tasks, scalability to large datasets, and provision of detailed insights into OOD inputs.

To further elevate the capabilities of MCM, we introduced prompt learning, a fine-tuning method that enabled us to tailor the model to specific datasets, thereby enhancing its

performance. Although our computational resources limited us to training the model for 200 epochs, we anticipate that longer training durations could yield even more promising results. Our combined approach of prompt learning + MCM demonstrated a noticeable improvement of approximately 1-2% over MCM alone, reinforcing the significance of prompt learning in OOD detection.

Looking ahead, we hope our research will serve as a stepping stone for future studies in the realm of multi-modal Deep Learning. Our exploration into MCM and prompt learning opens up exciting avenues for further advancements in OOD detection techniques, providing new perspectives for tackling challenges in the open world of machine learning systems.

References

- [1] S. Sali, “Exploring Multi-Modal Deep Learning,” Jun. 2023. [Online]. Available: <https://github.com/sid4sal/LanguageGuidedVision>
- [2] Kaya and Bilge, “Deep metric learning: A survey,” *Symmetry*, vol. 11, no. 9, p. 1066, Aug 2019. [Online]. Available: <http://dx.doi.org/10.3390/sym11091066>
- [3] X. Wang, X. Han, W. Huang, D. Dong, and M. R. Scott, “Multi-similarity loss with general pair weighting for deep metric learning,” 2020.
- [4] K. Roth, O. Vinyals, and Z. Akata, “Integrating language guidance into vision-based deep metric learning,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2022, pp. 16 177–16 189.
- [5] S. Sali, “Language Guidance for DML,” 2022. [Online]. Available: <https://github.com/sid4sal/MultimodalDML>
- [6] Y. Geifman and R. El-Yaniv, “Selectivenet: A deep neural network with an integrated reject option,” 2019.
- [7] Y. Ming, Z. Cai, J. Gu, Y. Sun, W. Li, and Y. Li, “Delving into out-of-distribution detection with vision-language representations,” in *Advances in Neural Information Processing Systems*, A. H. Oh, A. Agarwal, D. Belgrave, and K. Cho, Eds., 2022. [Online]. Available: <https://openreview.net/forum?id=KnCS9390Va>
- [8] K. Zhou, J. Yang, C. C. Loy, and Z. Liu, “Learning to prompt for vision-language models,” *International Journal of Computer Vision*, vol. 130, no. 9, pp. 2337–2348, Sep 2022. [Online]. Available: <https://doi.org/10.1007/s11263-022-01653-1>
- [9] —, “Conditional prompt learning for vision-language models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2022, pp. 16 816–16 825.
- [10] S. Bell and K. Bala, “Learning visual similarity for product design with convolutional neural networks,” *ACM Trans. Graph.*, vol. 34, no. 4, jul 2015. [Online]. Available: <https://doi.org/10.1145/2766959>

- [11] C. Huang, C. C. Loy, and X. Tang, “Unsupervised learning of discriminative attributes and visual representations,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 5175–5184.
- [12] H. Oh Song, Y. Xiang, S. Jegelka, and S. Savarese, “Deep metric learning via lifted structured feature embedding,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016.
- [13] F. Schroff, D. Kalenichenko, and J. Philbin, “Facenet: A unified embedding for face recognition and clustering,” in *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 815–823.
- [14] J. Wang, Y. Song, T. Leung, C. Rosenberg, J. Wang, J. Philbin, B. Chen, and Y. Wu, “Learning fine-grained image similarity with deep ranking,” in *2014 IEEE Conference on Computer Vision and Pattern Recognition*, 2014, pp. 1386–1393.
- [15] J. R. Hershey, Z. Chen, J. Le Roux, and S. Watanabe, “Deep clustering: Discriminative embeddings for segmentation and separation,” in *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2016, pp. 31–35.
- [16] D. Bouchacourt, R. Tomioka, and S. Nowozin, “Multi-level variational autoencoder: Learning disentangled representations from grouped observations,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.
- [17] Y. Movshovitz-Attias, A. Toshev, T. K. Leung, S. Ioffe, and S. Singh, “No fuss distance metric learning using proxies,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Oct 2017.
- [18] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, “Arcface: Additive angular margin loss for deep face recognition,” in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 4685–4694.
- [19] J. Lu, J. Hu, and Y.-P. Tan, “Discriminative deep metric learning for face and kinship verification,” *IEEE Transactions on Image Processing*, vol. 26, no. 9, pp. 4269–4282, 2017.
- [20] P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, and D. Krishnan, “Supervised contrastive learning,” in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 18 661–18 673. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/file/d89a66c7c80a29b1bdbab0f2a1a94af8-Paper.pdf>
- [21] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, “Momentum contrast for unsupervised visual representation learning,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.

- [22] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A simple framework for contrastive learning of visual representations,” in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, H. D. III and A. Singh, Eds., vol. 119. PMLR, 13–18 Jul 2020, pp. 1597–1607. [Online]. Available: <https://proceedings.mlr.press/v119/chen20j.html>
- [23] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, “Learning transferable visual models from natural language supervision,” in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, M. Meila and T. Zhang, Eds., vol. 139. PMLR, 18–24 Jul 2021, pp. 8748–8763. [Online]. Available: <https://proceedings.mlr.press/v139/radford21a.html>
- [24] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” *arXiv preprint arXiv:1810.04805*, 2018.
- [25] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “Roberta: A robustly optimized bert pretraining approach,” *arXiv preprint arXiv:1907.11692*, 2019.
- [26] K. Roth, T. Milbich, B. Ommer, J. P. Cohen, and M. Ghassemi, “S2sd: Simultaneous similarity-based self-distillation for deep metric learning,” 2021.
- [27] D. Hendrycks and K. Gimpel, “A baseline for detecting misclassified and out-of-distribution examples in neural networks,” 2018.
- [28] S. Liang, Y. Li, and R. Srikant, “Enhancing the reliability of out-of-distribution image detection in neural networks,” 2020.
- [29] O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar, “Cats and dogs,” in *2012 IEEE Conference on Computer Vision and Pattern Recognition*, 2012, pp. 3498–3505.
- [30] C. Wah, S. Branson, P. Welinder, P. Perona, and S. Belongie, “The caltech-ucsd birds-200-2011 dataset,” California Institute of Technology, Tech. Rep. CNS-TR-2011-001, 2011.
- [31] J. Krause, M. Stark, J. Deng, and L. Fei-Fei, “3d object representations for fine-grained categorization,” in *2013 IEEE International Conference on Computer Vision Workshops*, 2013, pp. 554–561.
- [32] L. Bossard, M. Guillaumin, and L. Van Gool, “Food-101 – mining discriminative components with random forests,” in *Computer Vision – ECCV 2014*, D. Fleet,

- T. Pajdla, B. Schiele, and T. Tuytelaars, Eds. Cham: Springer International Publishing, 2014, pp. 446–461.
- [33] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “Imagenet: A large-scale hierarchical image database,” in *2009 IEEE Conference on Computer Vision and Pattern Recognition*, 2009, pp. 248–255.
- [34] R. Huang and Y. Li, “Mos: Towards scaling out-of-distribution detection for large semantic space,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2021, pp. 8710–8719.
- [35] S. Fort, J. Ren, and B. Lakshminarayanan, “Exploring the limits of out-of-distribution detection,” in *Advances in Neural Information Processing Systems*, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, Eds., 2021. [Online]. Available: <https://openreview.net/forum?id=j5NrN8ffXC>
- [36] W. Liu, X. Wang, J. Owens, and Y. Li, “Energy-based out-of-distribution detection,” in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 21 464–21 475. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/f5496252609c43eb8a3d147ab9b9c006-Paper.pdf
- [37] D. Hendrycks and K. Gimpel, “A baseline for detecting misclassified and out-of-distribution examples in neural networks,” in *International Conference on Learning Representations*, 2017. [Online]. Available: <https://openreview.net/forum?id=Hkg4TI9xl>

Appendix A

Appendix: DML

In our evaluation of deep metric learning, we make use of three widely recognized benchmark datasets commonly employed in this field. These datasets include CARS196 and CUB200-2011, which are both more compact datasets, as well as Stanford Online Products, which is a large-scale dataset. To assess the performance in retrieval and re-identification tasks, we report two important metrics: Recall@k and f1 Score. These metrics provide valuable insights into the model’s ability to retrieve and accurately identify relevant data points, contributing to a comprehensive evaluation of the deep metric learning approaches applied in this study.

Stanford Online Products (SOP) is an image collection of products downloaded from eBay.com. For training, 11,318 classes (59,551 images) out of 22,634 classes are used, and 11,316 classes (60,502 images) are held out for testing.

CARS196 contains 196 different types of cars distributed over 16,185 images. The first 98 classes (8,054 images) are used for training and the other 98 classes (8,131 images) for testing.

CUB200-2011 consolidates images of 200 different bird species with 11,788 images in total and is an extended version of the CUB200 dataset. The first 100 classes (5,864 images) are used for training and the second 100 classes (5,924 images) for testing

Appendix B

Appendix: OOD Detection

Prior research on out-of-distribution (OOD) detection has predominantly concentrated on small-scale datasets, such as CIFAR and TinyImageNet, containing low-resolution and blurry images. However, with the emergence of pre-trained models like CLIP, the possibilities for extending OOD detection to more realistic and intricate datasets have expanded.

We consider a diverse set of in-distribution (ID) datasets, comprising **CUB-200**, **STANFORD-CARS**, **FOOD-101**, **OXFORD-PET**, and various **IMAGENET** variations. For OOD tests, we employ the same datasets utilized in previous studies, consisting of subsets from **iNaturalist**, **SUN**, **PLACES**, and **TEXTURE**. It is important to note that the categories in the OOD datasets do not overlap with those in the ID dataset.

For analysis, we make use of subsets of ImageNet-1k. we create **ImageNet-10**, which replicates the class distribution of CIFAR-10 but incorporates high-resolution images. Additionally, to enable rigorous OOD evaluation, we curate **ImageNet-20**, comprising 20 classes semantically similar to those in ImageNet-10 (e.g., dog (ID) vs. wolf (OOD)). By encompassing larger-scale datasets with varying resolutions, diverse content, and an increased number of classes, our aim is to offer a comprehensive and extensive evaluation of OOD detection using pre-trained models like CLIP. This will enable us to assess the generalizability and robustness of OOD detection methods across different real-world scenarios, providing valuable insights for OOD detection in complex and challenging settings.

Hard Prompts: Hard prompts are fixed, hand-engineered text prompts. These prompts are pre-defined and do not change during the training process.

Soft Prompts: Soft prompts are learnable text prompts utilized in prompt learning or fine-tuning language models. Unlike hard prompts, soft prompts are not fixed and can be adjusted during the training process. They are represented as vectors and learned alongside the model’s parameters. Soft prompts offer more flexibility and can be dynamically optimized, leading to improved model performance and better adaptability to various tasks and datasets.